

智能家居中增强型触发-动作编程规则的检索增强生成

赵宇辰*, 东南大学计算机科学与工程学院, 中国 王立夫*, 东南大学计算机科学与工程学院, 中国 董凯†, 东南大学计算机科学与工程学院, 中国

触发-动作编程是智能家居自动化的一种流行范式, 允许用户以“如果触发器, 则动作”的形式创建规则。虽然大语言模型为从自然语言生成TAP规则提供了一条有前景的路径, 但随着平台演进以支持增强型TAP规则, 其原始应用(例如仅依赖预训练知识和基础提示)便显得力不从心。此类规则融合了用于条件逻辑、计算和外部API调用的脚本。增强型TAP规则要求用户表达复杂的逻辑和环境上下文, 这使得没有深厚编程背景的用户难以创建此类规则。本文介绍了HomeGenii, 一个检索增强生成系统, 用于自动化增强型TAP规则的创建。HomeGenii构建了一个紧凑且具代表性的规则库, 使用先聚类后搜索方法检索语义对齐的规则, 并应用压缩技术以最小化令牌开销。评估显示, HomeGenii将增强型TAP规则生成的准确率提升至84%, 相比未使用RAG的系统提高了70%。我们的工作展示了一条可行的途径, 使非专业用户能够利用大语言模型实现富有表现力且复杂的家庭自动化。

CCS概念: • 以人为中心的计算 → 普适与移动计算。

其他关键词和短语: 家庭自动化, TAP生成, 检索增强生成, 大语言模型

ACM参考格式:

赵宇辰、王立夫和董凯。2026。智能家居中增强型触发-动作编程规则的检索增强生成。《ACM交互式、移动、可穿戴与普适技术汇刊》第10卷, 第1期, 文章26 (2026年3月), 32 页。 <https://doi.org/10.1145/3789673>

1 引言

触发-动作编程 (TAP) 已成为智能家居自动化的事实标准范式 [56], 被IFTTT [21]、SmartThings [41], 和Home Assistant等平台广泛采用 [17]。用户可以创建TAP规则, 通过遵循简单的“如果触发器, 则动作”结构来自动化任务。为了改善创建TAP规则的用户体验, 先前的研究已利用多样化的信息源来实现自动化规则生成。这些信息源包括手势和声音等现场交互 [32, 35]、历史事件日志 [30, 55], 以及自然语言描述 [33, 52]。最近, 大语言模型 (LLMs) 的快速发展

*作者对本研究的贡献相同。

†通讯作者。

作者联系方式: 赵宇辰, 东南大学计算机科学与工程学院, 江苏南京, 中国, yuchenzhao@seu.edu.cn; 王立夫, 东南大学计算机科学与工程学院, 江苏南京, 中国, wanglf@seu.edu.cn; 董凯 (通讯作者), 东南大学计算机科学与工程学院, 江苏南京, 中国, dk@seu.edu.cn。



本作品采用知识共享署名 4.0 国际许可协议进行许可。© 2026 版权所有所有者/作者所有。ACM 2474-9567/2026/3-ART26<https://doi.org/10.1145/3789673>

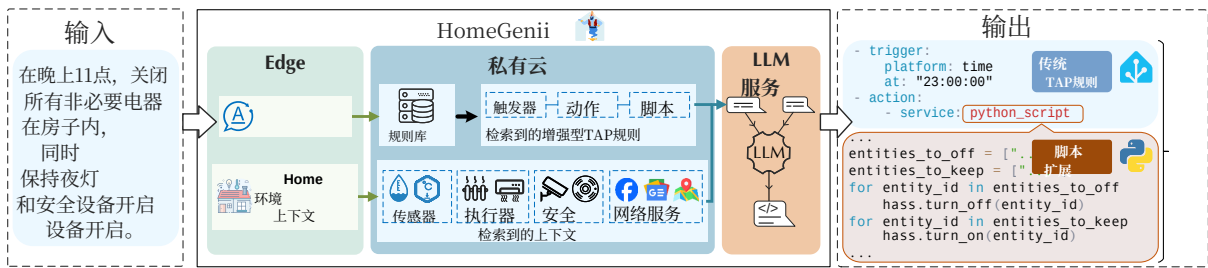


图 1. HomeGenii 使用。给定一个自然语言查询，HomeGenii利用检索到的增强型TAP规则和上下文信息来指导可执行脚本的创建。

激发了对基于LLM的方法来解读用户意图的日益增长的兴趣 [13, 26]。这种方法为家庭自动化提供了一条更具表现力的途径。

然而，随着主流平台通过超越简单触发器和动作的脚本能力扩展TAP（我们在本文中称之为增强型触发-动作编程） [18, 20, 53]，基于大语言模型的方法面临着新的挑战。现有基于大语言模型的方法主要依赖预训练知识，缺乏有效生成此类规则的必要信息。我们的形成性研究证实，有效的提示设计不仅需要编码显式的控制逻辑（例如，计算、循环、条件语句），还需要编码特定的环境上下文（例如，设备能力、传感器数据）。然而，这种对类编程精度的需求，从根本上与TAP的原始目标相矛盾：TAP旨在赋能广泛的非专业用户，而不仅仅是专业人士。因此，我们的研究探讨了这个问题：非专业用户如何利用大语言模型为复杂场景自动生成增强型TAP规则？

在本文中，我们提出HomeGenii，这是一个利用检索增强生成（RAG）通过检索专家编写的规则和相关环境上下文来生成增强型TAP规则的系统。图1展示了HomeGenii的使用方式。它首先在边缘端接受用户的自然语言查询。该查询连同家庭上下文随后被发送到私有云，在那里执行检索过程。此过程查询一个由专家编写的TAP规则组成的规则库，以找到最相关的增强型TAP示例，同时收集相关的环境上下文。检索到的信息随后作为精心构建的提示的一部分输入到LLM服务中。最后，基于此检索到的知识，大语言模型生成最终可执行的增强型TAP规则，并以所需的语法（例如，作为Home Assistant的Python脚本）输出，以实现用户意图的自动化。

尽管RAG方法前景广阔，但要将其有效应用于增强型TAP规则生成，我们必须审慎应对三个挑战。第一个挑战在于构建一个能全面覆盖广泛用户需求的规则库。理想的规则库应包含与每个可能的用户请求完全匹配的条目，但用户需求的开放性和多样性使得通过持续扩展来覆盖所有场景是不切实际的。第二个挑战源于从模糊查询中进行检索时存在的语义鸿沟。非专业人士在描述复杂场景时常常遗漏关键细节，这使得检索变得困难。他们可能提及设备类型而非具体型号（例如，“一个温度传感器”而非某个特定传感器），描述目标而非具体步骤（例如，“让房间更凉爽”，这可能涉及空调或开窗），或者省略参数值（例如，24°C的目标温度）。第三个挑战涉及增强提示所带来的开销。用检索到的材料增强提示不可避免地会增加令牌使用量。随着检索到的信息规模增大，每个提示的总令牌数量也会增加，从而带来额外的经济负担。

为应对这些挑战，我们在HomeGenii中结合领域洞察设计了多种方法。针对第一个挑战，我们设计了一个从基础安全到个性化的四层分层分类法，

场景来指导TAP规则库的构建。在此分类法的指导下，HomeGenii将高层意图解构为规范的触发-行为对，将其映射到可重用的增强型TAP规则设计模式，并最终将具体实例抽象为一组参数化模板。针对第二个挑战，我们在HomeGenii中引入了一种先聚类后搜索的TAP规则检索方法。HomeGenii使用两种表示方式对规则库进行聚类：行为语义和词汇语义。给定一个用户查询，HomeGenii随后识别最相关的聚类，并对该聚类内的候选规则进行排序，以检索最相关的TAP规则。针对第三个挑战，我们在HomeGenii中采用了一种压缩与验证方法来减少不必要的令牌消耗。HomeGenii通过语义锚定保留关键语义，并对检索到的规则应用多轮处理进行小型化。为了减轻LLM响应中可能出现的幻觉，HomeGenii包含了一个带有自校正循环的自动化验证系统，以确保符合目标平台的模式规范，并执行用户参与循环验证以确保与用户意图保持一致。

我们在HomeGenii上进行了全面评估。为评估规则生成质量，我们报告了准确率（生成的规则是否通过所有单元测试并端到端执行）和CodeBLEU（与真实规则的相似度）。首先，我们评估系统生成增强型TAP规则的能力。HomeGenii将准确率从14%（无RAG）提升至约84%。接下来，我们使用来自IFTTT网站的推荐小程序评估规则库分类法的构建，重点关注分类法的正交性和数据集覆盖率。然后，我们检查对专家规则库进行聚类的效果。语义聚类使生成准确率相对于未聚类的基线提高了约27%，并将检索成功率提升了约16%。我们进一步评估了上下文检索性能，发现系统实现了约91%的F1分数。我们的压缩方法在保留完整功能行为的同时，将令牌使用量减少了约30%。我们还包含了一个案例研究，追踪HomeGenii如何通过聚类、检索、压缩和验证阶段处理用户请求，以生成可执行的增强型TAP规则。最后，我们邀请了15名参与者使用HomeGenii在复杂场景中生成规则。结果表明，用户对系统有高度信任、良好的可解释性、处理模糊查询的能力，并愿意长期继续使用该系统。

总而言之，我们做出了以下贡献：

- 我们解决了一个实际的智能家居问题：使非专家能够有效利用主流平台发布的基于脚本的TAP扩展，以处理更复杂的自动化场景。
- 我们首次将RAG应用于TAP生成。通过对规则库进行聚类，并检索相关的TAP规则及其关联的环境上下文，我们为大语言模型补充了所需的信息。
- 我们设计了一套用于实现有效RAG的技术：(1) 用于可扩展规则库构建的层次分类法，(2) 用于弥合语义鸿沟的先聚类后搜索检索机制，以及 (3) 用于减少令牌开销的压缩策略。
- 我们实现了HomeGenii系统，并展示了其在增强型TAP规则生成方面的强大性能。

2 背景与相关工作

在本节中，我们首先介绍触发-动作编程（TAP）检索增强生成（RAG技术的背景。然后，我们回顾关于TAP规则自动生成的相关研究。

2.1 背景

TAP与增强型TAP。TAP是一种简化的事件驱动范式，允许用户通过创建“如果 触发器，则 动作”规则来自动化任务。触发器是特定事件，例如传感器激活；而动作是随之执行的操作，例如打开设备。例如，在规则“如果检测到运动，则打开灯”中，运动检测是触发器，它会自动执行点亮灯具的动作。

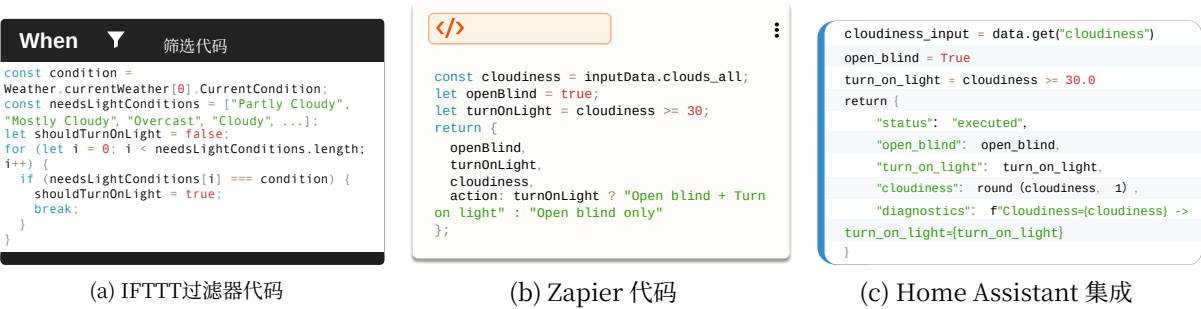


图 2. 针对复杂场景，三个自动化平台上的增强型触发-动作编程示例。

近来，为满足处理日益复杂的自动化场景的需求，主流家庭自动化平台已开始通过将脚本代码整合到自动化规则中，来扩展经典的 TAP 范式。本文中此类扩展范式称为增强型 TAP。如图2所示，三个自动化平台通过不同的实现方式展示了这一演进。IFTTT过滤器代码 [20] 在触发器与动作之间插入一段 JavaScript 代码片段。此代码充当可编程过滤器，访问触发器数据以执行逻辑检查，并有条件地停止规则执行（例如，通过 `service.skip()`），从而实现更精细的控制。类似地，Zapier 代码 [53]允许用户将自定义的 Python 或 JavaScript 直接嵌入自动化工作流中。Home Assistant Python 脚本集成 [18] 允许沙盒化的 Python 文件直接作为自动化动作执行。结合其基于 YAML 的自动化和 Jinja2 模板，Python 脚本使用户能够定义复杂的、数据驱动的行为。虽然这些新引入的功能解锁了更强大的自动化，但也提高了创建自动化规则的门槛。

RAG 技术。 RAG 是一种通过将大语言模型连接到外部知识源来增强其功能的技术 [10]。RAG 系统并非仅依赖其内部训练数据，而是首先根据用户的查询，从指定的文档库中检索相关信息。检索方法大致可分为两种类型：稀疏检索（例如基于关键词的 BM25 [40]，）和稠密检索（后者将查询和文档嵌入向量空间以执行语义相似度搜索，例如密集段落检索器 [24]等模型）。另一个关键设计选择是检索粒度。虽然有些方法检索整个文档，但更常见的做法是检索较小的段落或块，以提高相关性和效率 [16, 27]。更细的粒度，例如词元 [25] 或实体 [12]，也已在专门任务中得到探索。在生成阶段，模型基于检索到的信息和输入查询生成文本。此过程可能涉及参数可访问的白盒生成器（如 T5 [46]，）或参数不可访问的黑盒生成器（如 GPT [1]）。增强过程通过三种不同方式将检索到的信息整合到生成流程中：将检索到的文档与查询作为输入进行拼接 [22, 37, 43]，；将信息注入模型的中间层级 [3, 12, 49]，；或对输出进行后处理以更好地反映检索到的内容 [15, 25, 57]。在本文中，我们首次将 RAG 应用于规则生成，以处理复杂的自动化场景。

2.2 相关工作：家庭自动化规则生成

已有多种TAP生成方法被提出，旨在提升创作自动化的用户体验。如表1所示，我们将其分为三类：原位编程、轨迹驱动方法以及基于查询的生成。

表1. 现有TAP生成方法对比。

Type	系统	方法	免训练?	模糊描述	增强型TAP
原位	SLP [42]	情境化实时编程	N/A	×	×
	Wizard-of-Oz [32]	快照与模拟	N/A	×	×
轨迹驱动	Trace2TAP [55]	SAT求解	N/A	×	×
	SGen [30]	数据挖掘	N/A	×	×
	ASGen [44]	Transformer解码器	×	×	×
基于查询的	Helion [33]	N元语法	×	×	×
	RecipeGen [52]	Transformer序列到序列模型	×	×	×
	Sasha [26]	LLMs	✓	✓	×
	ChatIoT [13]	LLMs	✓	×	×
	HomeGenii (我们的方法)	RA-LLMs	✓	✓	✓

原位编程。基于TAP框架，原位编程范式允许用户在智能环境 [32]中，利用自然交互和上下文信息来配置自动化规则。该范式的特点是：(1) 融合了普适的多模态界面，支持通过语音、手势、触控和姿势进行交互；(2) 利用了上下文信息，如时间、位置、设备状态和用户行为。例如，Wizard-of-Oz [32] 使非专家用户能够利用自然的多模态交互和上下文数据来配置智能家居自动化规则。类似地，SLP [42] 采用了与原位编程相关的方法。它引入了情境化实时编程，这是一种专为人机协作设计的范式，使非专家用户能够为需要响应性和协调性的任务编程协作机器人。虽然原位编程扩展了TAP的交互模态，但仅靠原位交互通常难以在复杂的自动化场景中表达所需的控制逻辑。

轨迹驱动方法。轨迹驱动方法基于历史数据（如传感器读数和设备动作）生成自动化规则。例如，Trace2TAP [55] 利用带时间戳的传感器读数和设备动作日志，采用符号推理和SAT求解 [8] 来生成、聚类和排序规则。SGen [30]应用数据挖掘技术，为事件触发和时间触发场景合成规则，识别频繁的事件对和时间模式。ASGen [44] 利用Transformer解码器来整合事件属性和外部知识以进行规则生成。然而，轨迹驱动方法一方面剥夺了用户定义自动化的自主权，使得系统不够完全可控；另一方面，严重依赖于模型的准确率。

基于查询的方法。基于查询的方法利用语言模型来解释自然语言输入并生成相应的TAP规则。早期方法通常依赖于较小的、针对特定任务的模型。例如，Helion [33], 基于自然语言的可预测性，采用n元语言模型来识别用户驱动序列中的语义模式，从而促进生成现实的自动化场景。RecipeGen [52]利用Transformer序列到序列模型将用户描述翻译成TAP，以精细的粒度处理功能级（通道和功能）和字段级（特定参数）的细节。随着生成能力的进步，最近的方法越来越多地采用大语言模型（LLMs）进行TAP规则生成。这些方法通常不需要对大型基础模型进行额外训练，而是专注于有效的提示或上下文工程。Sasha [26] 采用LLMs来推断用户目标，并通过创建和执行行动计划或自动化例程来处理模糊描述。它引入了一个迭代推理过程，包括澄清用户目标和可行性、过滤相关设备、规划实现目标的自动化规则、将计划映射到适当的API调用，以及根据用户反馈优化结果。ChatIoT [13] 同样利用LLMs来解释用户请求

并自动生成TAP。它采用了一种上下文感知压缩提示策略，以减少令牌消耗，同时提高生成准确率。它还通过从多模态大语言模型中进行知识蒸馏来创建新的人工智能模型，并利用一种新颖的深度强化学习方法进行模型定制。虽然现有系统在处理模糊描述或令牌效率等特定挑战方面取得了显著进展，但其生成增强型TAP规则的能力仍然有限。为了解决这一局限，我们的系统HomeGenii引入了一种新颖的检索增强生成框架。该框架能有效管理模糊描述以生成增强型TAP，同时保持可控的计算开销。

3 形成性研究及分析

为引导本研究，我们首先考察以下研究问题：用户提供的提示中控制逻辑细节和环境上下文的缺失，在多大程度上对生成的结果产生负面影响？

3.1 研究设计

3.1.1 任务选择：复杂场景。我们基于文档[19]设计了一组20个复杂场景。选择这些任务是因为它们需要增强型TAP规则的高级功能，例如条件逻辑、数值计算以及外部服务调用。一个示例任务是：“当我说‘早上好’时，如果是工作日，播报今日天气和我日历上的第一个事件。如果是周末，则提供周末问候并播放Spotify上的推荐播放列表。”此任务需要条件分支和外部API调用。

3.1.2 变量操纵：提示构建。我们根据为控制逻辑和环境上下文提供的细节程度，设计了四种提示变体。

1) 最小提示：一个高层次的任务描述，反映典型的用户查询。例如，“当我说早上好时，告诉我天气和我的日程安排”。

2) 丰富上下文：在最小提示基础上，增加了详细的环境上下文（例如，设备列表、应用程序接口）。例如，“...可用设备包括：‘小爱’智能音箱、Weather.com 应用程序接口、Google 日历应用程序接口...”。**3) 丰富逻辑：**在最小提示基础上，增加了显式的控制逻辑（例如，伪代码或分步规则）。例如，“...首先，检查 `datetime.now().weekday() < 5`；如果是，则调用 `weather_api.get_today()` ...”。**4) 丰富逻辑 + 丰富上下文：**理想的提示，包含详细的上下文和显式的逻辑。

3.1.3 实施设置。我们采用以下评估设置。

大语言模型。我们选择了三个商业化的最先进模型：*GPT-5*、*Gemini- 2.5 Pro*和*DeepSeek-R1*。除非另有说明，解码采用贪婪策略（温度 = 0；top- p = 1； n = 1）。我们提供了一个简短的系统头部，将输出模式固定为“仅代码”的IFTTT过滤器代码风格，以确保可解析性。

数据集。针对20个场景中的每一个，我们实例化了上述四种提示变体，总共生成80个提示。真实值由两位具备领域专业知识的博士研究生独立编写的增强型TAP规则构成，并以IFTTT过滤器代码格式表示。作者们进行了交叉评审并协调了所有分歧；最终成果作为参考规则和单元测试预言。

执行测试框架。所有生成的程序都在一个模拟IFTTT过滤代码运行时的沙箱中执行：外部API（例如，天气、日历、音乐）被替换为确定性测试夹具；时间和区域设置被模拟；设备状态也被模拟。该框架为每个场景暴露一组任务特定单元测试（涵盖工作日/周末分支、API故障、空日历等），并记录函数调用和返回值。无法解析或违反输出模式的代码被视为执行失败。

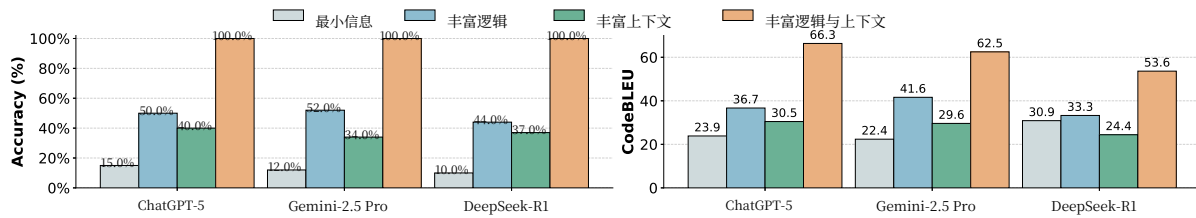


图3. 形成性研究结果，比较了ChatGPT-5、Gemini-2.5 Pro和DeepSeek-R1在四种信息条件（最小信息、丰富逻辑、丰富上下文、丰富逻辑与上下文）下的准确率和CodeBLEU得分。

指标。我们报告两个主要指标。第一个指标是准确率。即模型生成的单个程序在沙箱中通过所有单元测试的场景比例。该指标衡量代码在指定条件下是否能端到端正确运行。第二个指标是CodeBLEU [39]。即生成的程序与真实值程序之间的相似度，结合了 n -gram重叠度（BLEU）、加权关键词、语法/抽象语法树线索以及简单的语义匹配。我们在移除注释和格式化差异后，对标准化的JavaScript（IFTTT过滤器代码的语言）计算CodeBLEU。

3.1.4 实验方法。针对每个场景和每个模型，我们提交上述定义的四种提示变体。每个输入在固定架构下生成一个单一程序；输出首先会验证其可解析性，然后在沙箱中针对该场景的单元测试执行。

3.2 结果与分析

结果如图3所示。在提示中丰富逻辑细节（丰富逻辑）和任务特定上下文（丰富上下文）显著提高了流行大语言模型的执行准确率和代码质量。同时提供这两种信息（丰富逻辑和上下文）产生了更显著的效果，实现了100%的准确率以及60.8的平均CodeBLEU分数。因此，我们将研究问题重新表述为：给定用户的输入，我们如何获取这两类补充信息？

4 HomeGenii

在本节中，我们介绍HomeGenii，这是一个为生成增强型TAP规则而设计的基于RAG的系统。我们首先概述HomeGenii的用法，然后深入探讨实现该系统过程中遇到的关键挑战。

4.1 使用方式

HomeGenii的使用方式如图1所示。给定用户的自然语言请求后，HomeGenii首先并行地从专家策划的规则库中检索语义对齐的规则，并从上下文数据库中检索相关的环境上下文。然后，这些检索到的信息被整合到一个结构化提示中，该提示为大语言模型提供了必要的逻辑和参数。最终输出的是语法正确且具有上下文感知的增强型TAP规则，可直接在主流的平台执行。

4.2 挑战

尽管利用RAG来增强TAP生成看似直接，但在实践中却带来了三大挑战。

挑战1：规则库覆盖度困境源于多样化的用户需求。RAG方法依赖于一个高质量的增强型TAP规则外部数据库，但这样的数据库目前并不存在。理想情况下，规则库

应能包含与每个可能的用户查询完全匹配的条目。然而，考虑到用户需求的巨大多样性和复杂性，持续扩展规则库以覆盖所有潜在场景是不切实际的。因此，主要挑战在于如何在全面覆盖与保持规则库紧凑高效之间取得平衡。

解决方案：为构建一个具有代表性的增强型TAP规则库，我们首先引入一个从安全到个性化的四层分层分类法，以确保全面覆盖自动化意图。这些意图随后被提炼为触发-行为对，映射到五种规范设计模式，并合成为具体的规则实例。最后，这些实例被抽象为参数化模板。该方法通过模式复用，在保持规则库紧凑性的同时，实现了广泛的语义覆盖。详细过程概述于[第5.1节](#)。

挑战二：从模糊查询中检索的语义鸿沟。检索假定对场景有精确描述，然而非专业用户的请求在多个方面是模糊的。首先，用户经常省略具体的实体引用（例如，“卧室窗户传感器”）。其次，他们的意图可能不明确（例如，“安静一点”暗示了多种动作）。第三，必要的阈值经常未被指定（例如，“温度很高”）。最后，用户不会引用所需的应用程序接口或功能，这进一步扩大了其请求与自动化规则之间的差距。

解决方案：我们采用一种两阶段的先聚类后搜索检索策略，以提高在模糊查询下的精确率。首先，我们根据语义特征对功能相似的TAP规则进行聚类，形成连贯的组。其次，在查询时，我们通过相似度评分将用户请求路由到最相关的簇，并在该簇内执行细粒度的代码检索。基于顶部的TAP示例，我们随后检索必要的环境上下文（例如，设备、位置），以最大化执行可行性。具体细节在[第5.2节](#)和[第5.3节](#)中。

挑战3：增强提示带来的开销。用完整的检索到的信息增强提示，不可避免地会增加输入令牌数量。这会导致更高的令牌成本，并增加超出模型有限上下文窗口的风险，可能引发静默截断和生成失败。如果生成失败且系统重试，则会再次产生相同的预填充计算开销。

解决方案：我们最小化输入端的令牌开销，并减少输出端的重新生成。首先，为了在不改变其语义的情况下压缩检索到的TAP规则，我们保留语义关键的API调用，同时将非必要的语法元素修剪为紧凑的声明性中间表示，然后将其输入给大语言模型。其次，基于大语言模型的输出，我们引入一个自动化的检查与纠正机制，根据目标平台的要求验证结果。只有在代码通过此技术验证后，最终结果才会呈现给用户。更多细节在[第5.4节](#)和[第5.5节](#)中提供。

5 系统设计

在本节中，我们将介绍HomeGenii的系统设计。如[图4](#)所示，HomeGenii 包含五个组件：

- **规则库构造器** 通过层次分类法组织自动化意图，并基于这些分类后的自动化目标构建结构化的规则库。
- **语义聚类器** 利用行为语义和词汇语义表示，对规则库中功能相似的规则进行分组。
- **信息检索器** 在聚类规则库中定位相关簇并检索相应的规则。此外，它还利用大语言模型获取相关的环境上下文。
- **压缩增强器** 对检索到的规则应用一系列语义保持转换，创建一个为LLM消费优化的微型表示。

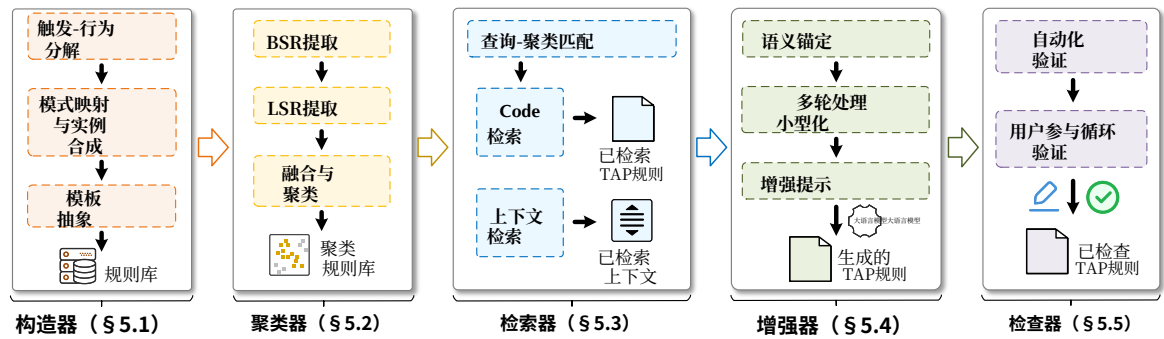


图 4. HomeGenii 概览。

表2. 家庭自动化目标的分层框架。

层级	核心目标	构成
1. 环境安全与安防	保护居住者和财产，并维护居住环境的完整性。	• 物理安全：入侵检测、警报器、视频录制。 • 消防安全：烟雾检测、火灾警报器、灭火系统。 • 应急响应：自动警报、疏散协议。
2. 资源优化与效率	减少浪费并提升能源与时间使用效率。	• 能源管理：能源节约与优化使用。 • 节省时间：自动化重复性家务。
3. 健康与福祉	主动营造一个居住环境以促进整体健康。	• 身体健康：空气质量监测与优化，水合提醒、睡眠环境优化。 • 心理健康：营造放松或专注的氛围。
4. 个性化与生活赋能	高度定制化的体验，赋能生活方式并提升日常生活品质。	• 场景与仪式：个性化预设（例如电影模式） • 娱乐与社交：媒体控制、派对模式。 • 个性化便利：高度定制化自动化（例如，阅读时自动灯光调节）。

- 规则检查 内核验证生成的TAP规则功能正确且符合预期与应用 针对特定错误进行有针对性的纠正。s,

5.1 规则库构造器

规则库构造器旨在为RAG构建一个紧凑的规则库。为确保此规则库覆盖多样化的用户需求，我们引入了一个层次分类法来对智能家居自动化意图进行分类。该分类法的灵感来源于马斯洛需求层次理论 [34]，并参考了关于智能家居中用户需求的研究 [7, 38, 51]。我们将心理层次转化为智能家居的语境，以确保覆盖范围从维持生命的基本需求延伸到更高层次的个人表达愿望。如表2所示，我们的分类法将自动化意图结构化四个递进的层级：环境安全与安防、资源优化与效率、健康与福祉以及个性化与生活赋能。

基于此分类法，规则库构造器随后将这些目标解构为触发-行为对（步骤一），将其映射到设计模式并合成实例（步骤二），最后将实例抽象为参数化模板（步骤三）。

步骤一：触发器-行为分解。在此步骤中，我们定义一组规范用户意图。随后，将每个意图分解为其基本计算组件，以形成一个触发器-行为对。

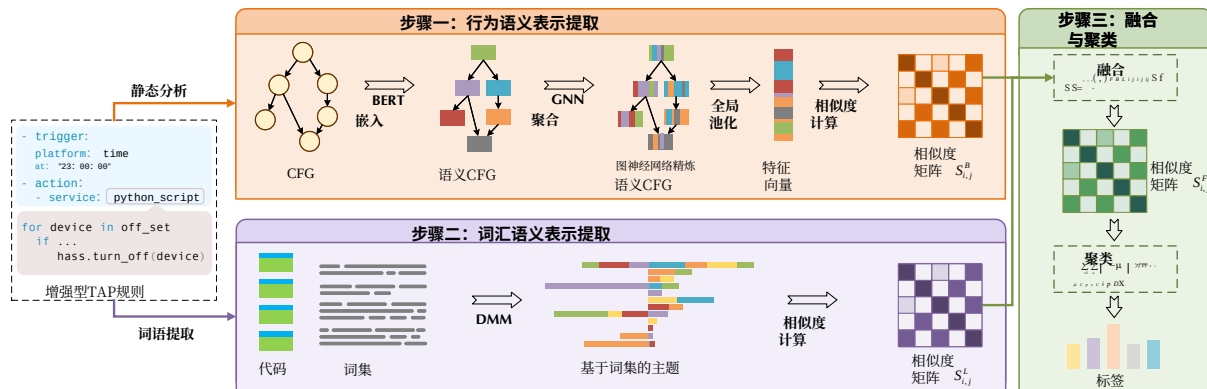


图5. 语义聚类器概览。

• 触发条件：规则的起始事件。其正式定义为一个能激活自动化的离散发生事件，例如传感器状态转换（如检测到运动₁）、时间事件（如日落时）、抽象系统状态变化（如房屋_模式切换至‘离家’），或外部应用程序接口调用。

• 预期行为：系统必须执行的一系列动作。这包括设备控制命令（例如，light.turn_on）、通知传递机制以及内部状态变量的更新。

步骤二：模式映射与实例合成。随后，触发器-行为对被映射到一组五个规范的增强型触发-动作编程设计模式。这些模式代表了实现超越简单触发器-动作映射的复杂自动化逻辑所必需的、反复出现的编程结构。这五个核心模式包括：(1) 状态机模式，用于管理高层级、互斥的系统模式（例如，在家、外出、睡眠、度假）之间的转换；(2) 轮询与超时模式，实现依赖于某个条件持续特定时长或在超时期限后失效的逻辑；(3) 复杂事件序列模式，捕捉需要多个事件按特定时间顺序发生才能触发动作的场景；(4) 数据聚合与计算模式，支持基于从多个数据源（例如平均温度或传感器的多数投票）计算出的指标进行决策；(5) 条件循环与迭代模式，支持对设备组进行批量操作或重复执行动作，直到满足终止条件。针对每个设计模式，我们合成了5-8个跨越我们目标分类法中不同上下文的具体规则实例。

步骤三：模板抽象。在这最后一步中，我们将具体的规则实例转换为参数化模板，以增强规则库的可重用性和可扩展性。此抽象过程识别并泛化实例中发现的设备特定标识符。例如，供应商特定的设备名称（如yeelight1s、kasa_bulb）被替换为通用参数 [灯]。这种转换允许单个模板代表一类规则，通过将参数绑定到用户环境中可用的特定设备，即可具体化这些规则。这些模板的总和构成了我们的规则库。

5.2 语义聚类器

虽然层次分类法为构建全面的规则库提供了原则性的结构，但每个类别标签下人工编写的规则实例在其词汇表达和实现细节上表现出显著差异。一个静态的、基于标签的检索系统会过于僵化，无法有效匹配用户多样且常常模糊的自然语言查询。因此，我们对

算法1：行为语义表示提取**输入：**控制流图 (N, E) **输出：**BSR**符号说明：** N : 基本块; E : 控制流图边。 V : 节点 (应用程序接口/控制); A : 类型化边 $t \in \{\text{顺序, 条件}\}$ 。 F_b, L_b : 首/末块 b 的节点锚点 ($\perp$ 若无)。 节点(标签, 类型): 创建节点。 边类型(u, v): 将控制流图边 (u, v) 映射到 t 。 BERT_编码、GNN_编码、全局池化: 节点文本嵌入、图编码和读出。

```

1  $V \leftarrow \emptyset; A \leftarrow \emptyset;$ 
2 对每个  $b \in N$  执行
3    $F_b \leftarrow \perp; L_b \leftarrow \perp;$ 
4 对每个  $b \in N$  执行
5   应用程序接口  $\leftarrow \text{ExtractAPIs}(b)$ ; 上一个  $\leftarrow \perp;$ 
6   如果 |应用程序接口| > 0 则
7     对于  $a \in \text{应用程序接口}$  (按顺序) 执行
8        $v \leftarrow \text{节点}(\text{标签} = a, \text{类型} = \text{"应用程序接口"})$ ;  $V \leftarrow V \cup \{v\};$ 
9       如果 上一个  $\neq \perp$  则
10         $A \leftarrow A \cup \{(\text{上一个} \rightarrow v, \text{"顺序"})\};$ 
11        如果  $F_b = \perp$  则
12           $F_b \leftarrow v;$ 
13           $\text{prev} \leftarrow v;$ 
14         $L_b \leftarrow \text{prev};$ 
15      否则如果  $\text{IsDecision}(b)$  那么
16         $v \leftarrow \text{节点}(\text{标签} = \text{获取\_条件}(b), \text{类型} = \text{"控制"})$ ;  $V \leftarrow V \cup \{v\}; F_b \leftarrow v; L_b \leftarrow v;$ 
17 对每个  $(u, v) \in E$  执行
18   if  $L_u \neq \perp \wedge F_v \neq \perp$  then
19      $t \leftarrow \text{边类型}(u, v)$ ;  $A \leftarrow A \cup \{(L_u \rightarrow F_v, t)\};$ 
20  $X \leftarrow \text{BERT\_编码}(\text{标签}(V)); H \leftarrow \text{图神经网络\_编码}(V, A, X); \text{BSR} \leftarrow \text{全局池化}(H);$ 
21 返回 BSR;

```

规则库进行基于语义的聚类。我们采用行为语义和词汇语义作为互补的表示方法，以全面捕捉TAP规则的含义。行为语义对规则的控制流和逻辑结构（例如，序列、循环、条件语句）进行建模，而词汇语义则捕捉表面层级的术语，例如设备名称和API标识符。这两种表示方法的结合，能够与用户表达需求的方式（词汇层面）以及系统实现需求的方式（行为层面）对齐，从而弥合模糊查询与精确程序逻辑之间的差距。如图5所示，语义聚类器提取行为语义表示（步骤一）和词汇语义表示（步骤二），然后整合这两种表示来对TAP规则进行聚类（步骤三）。

步骤一：行为语义表示（BSR）提取。此步骤旨在捕获TAP规则的功能行为。该过程首先使用既定的静态分析技术将TAP规则转换为控制流图（CFG）[2, 31]。然后，算法1将TAP规则的CFG编码为BSR。它首先初始化语义图容器，将节点集 V 和边集 A 设置为 $\emptyset$ （第1行），并为每个基本块 b 分配一个入口/出口锚点（第2行）。随后，它构建块内结构（第3–16行）：对于每个块，提取其顺序API列表并清空临时前驱prev。如果该块包含

API调用, 算法会为每个调用创建一个 api 节点并将其添加到 V ; 同一块内连续的调用通过顺序边($prev \rightarrow v$)连接到 A 。创建的第一个节点被记录为该块的入口锚点 F_b , 并更新 $prev$; 处理完该块后, 最后创建的节点成为出口锚点 L_b 。如果块没有API调用但是一个决策点, 该过程会创建一个带有谓词标签的单个控制节点, 将其添加到 V , 并将两个锚点都设置为此节点, 即 $F_b = L_b$ 。接下来, 将块间控制流投影到语义图上(第17–19行): 对于每条CFG边 (u, v) , 当两个锚点都存在时, 插入一条类型化边 $(L_u \rightarrow F_v, t)$, 其中 $t \in \{\text{顺序、条件}\}$ 由 $\text{EdgeType}(u, v)$ 决定。最后, 通过BERT_Encode嵌入节点标签, 使用GNN_Encode($V, A, \cdot$)传播结构信息, 并通过全局读出将节点嵌入聚合为单个向量 BSR (第20行)。最终, 我们计算规则库中所有BSR向量之间的两两余弦相似度, 以构建行为相似度矩阵。

步骤二: 词汇语义表示提取。此步骤捕获TAP规则中诸如设备名称和API标识符等离散的词级语义。这些术语频繁共现, 表明对应的规则更可能属于同一类别。

我们首先将函数名称分解为单个单词以构建词集。基于这些词集, 我们采用狄利克雷多项式混合模型[36]来建模TAP规则中的词分布。DMM模型专为文档生成而设计, 它假设每个文档由单个主题生成[14]。在此上下文中, 每个词集被视为一个文档, 而主题则对应潜在语义簇。每个混合成分 z 对应一个簇 k [50]。当生成一个文档时 d , DMM首先根据混合权重选择一个簇 k , 这些权重代表选择每个簇的可能性。然后, 文档 d 由所选簇根据其分布生成 k 。因此, 文档 $p(d|z = k)$ 的整体似然可以表示为所有可能簇的概率之和:

$$p(d) = \sum_{k=1}^K p(z = k) p(d | z = k), \quad (1)$$

其中 K 是簇的数量。为了定义 $p(d|z = k)$ 和 $p(z = k)$, DMM采用了一个朴素贝叶斯模型, 其中文档中的词在给定簇标签 k 的条件下是独立的, 并且其概率与它们在文档中的位置无关。因此, 文档 d 在给定簇 k 下的似然可以写作:

$$p(d|z = k) = \prod_{w \in d} p(w|z = k). \quad (2)$$

每个簇 k 被建模为一个词上的多项式分布, 使得 $p(w|z = k) = \phi_{k,w}$, 其中 $w = 1, \dots, V$ 且 $\sum_w \phi_{k,w} = 1$ 。每个混合成分(簇)的权重从一个多项式分布中采样, 使得 $p(z = k) = p(z = k|\Theta) = \theta_k$, 其中 $k = 1, \dots, K$ 且 $\sum_k \theta_k = 1$ 。

为了估计每个文档 d 的混合成分(簇) z , 我们利用吉布斯采样技术[28,50]。每个文档的簇分配 z 根据所有其他文档当前分配的条件后验分布进行迭代采样。在采样过程中, 参数 $\phi_{k,w}$ 和 θ_k 会根据当前的簇分配进行更新。基于估计的簇分配以及学习到的参数 $\phi_{k,w}$ 和 θ_k , 我们可以进一步量化不同词集(即不同的TAP规则)之间的相似度。具体来说, 每个文档 d 可以表示为一个在簇上的概率分布:

$$\mathbf{p}_d = [p(z = 1 | d), \dots, p(z = K | d)], \quad (3)$$

其中 $p(z = k | d) \propto p(d | z = k) p(z = k)$ 表示给定文档 d 时簇 k 的后验概率。随后, 两个文档 d_i 和 d_j 之间的相似度可以通过对

这些簇分布：

$$\text{sim}(d_i, d_j) = \frac{\mathbf{p}_{d_i} \cdot \mathbf{p}_{d_j}}{\|\mathbf{p}_{d_i}\| \|\mathbf{p}_{d_j}\|}. \quad (4)$$

两个T 具有相似API调用集或设备引用集的AP规则将表现出更高的相似度分数

s.

步骤三：表征 融合与聚类。在此步骤中，我们将BSR和LSR融合为单个特征

re

矩阵，然后对TAP规则进行聚类基于融合后的特征。

问题。直接拼接或组合这些表示是次优的，因为它们存在于根本不同的空间中：BSR是行为模式的连续向量表示，而LSR则被概括为源自基于聚类的词分布的成对规则相似性。简单的融合会模糊语义对齐的可解释性，并且无法有意义地整合这两种模式。

为了解决这个问题，我们提出了一种相似度级融合方法。我们首先在BSR和LSR各自的原生空间中计算相似度矩阵，然后在相似度级别进行融合。这种方法确保了行为模式和词汇语义首先在各自的原生空间中得到评估，然后才被整合以提供统一的相似度评估。

具体来说，对于BSR，我们计算BSR向量之间的余弦相似度以获得行为相似度矩阵。对于LSR，我们使用先前推导出的、表示词集之间词汇接近度的相似度矩阵。随后，我们采用注意力机制结合Adam优化器来学习一组自适应权重，从而得到一个融合相似度矩阵 s^F 。获得 s^F 后，我们将其转换为距离矩阵 D^F ，并在导出的度量空间上执行k均值聚类。形式上，令 $x_1, x_2, \dots, x_n$ 表示TAP规则， $C = C_1, C_2, \dots, C_k$ 为 k 个簇的集合。聚类的目标是基于融合后的距离最小化簇内失真：

$$\min_C \sum_{p=1}^k \sum_{x_i \in C_p} |x_i - \mu_p|_{D^F}^2, \quad (5)$$

其中 μ_p 表示聚类 C_p 在融合相似度空间中计算得到的质心。在实践中，此过程将TAP规则分组为语义连贯的聚类，这些聚类共同反映了其执行行为和词汇构成。

最后，我们采用一个自监督框架，该框架利用TAP规则本身的内在结构来训练一个分类器，用于将用户查询与推导出的聚类进行匹配。对于每个TAP规则样本 x_i ，我们提取设备和API函数名，并通过拆分复合标识符（例如，openCurtain $\rightarrow$ open, curtain）将其进一步分解为子词单元。然后，我们计算每个样本内的词频分布，并选择前四个最频繁出现的词：

$$W_i = w_{i,1}, w_{i,2}, w_{i,3}, w_{i,4}. \quad (6)$$

我们将这些词 W_i 输入到一个预训练的 *bert-base-uncased* 模型 [9] 中，该模型生成上下文化的嵌入向量 h_i 。然后， h_i 通过一个全连接分类层：

$$\hat{y}_i = \text{softmax}(W * h_i + b), \quad (7)$$

其中 W 和 b 表示分类器参数。模型训练使用Adam优化器进行，最小化预测标签 $\hat{y}_i$ 与簇分配 y_i 之间的交叉熵损失。

5.3 信息检索器

信息检索器首先在聚类规则库中搜索以定位相关规则（**步骤一**），随后搜索相关的上下文（**步骤二**）。

表3. 多遍小型化规则。

类别	转换规则	描述	示例
句法剪枝	声明关键字消除	令 $D \in \{\text{const, let, var}\}$: $T(D\ v = e;) \rightarrow v = e;$	<code>const temp = 25; → temp = 25;</code>
	标点最小化	对于语句 S : $T(S;) \rightarrow S_0$ 。 对于列表 $L = [e_1, \dots, e_n]$: $T(L) \rightarrow [e_1, \dots, e_n]$	<code>x = 5; → x = 5;</code> <code>[a,b,] → [a,b]</code>
	块内联	对于带有单个语句 S 的控制结构 C (例如 if, for) : $T(C(\text{条件})\{S\}) \rightarrow C(\text{条件})\ S$	<code>如果(x>0){y=1;} → 如果(x>0) y=1;</code>
声明式重构	迭代模式简化	给定 $\text{for}(i=0; i<\text{arr.len}; i++)$ 且循环体为 $B(i)$: $T(\cdot) \rightarrow \text{对于}(\text{item 的 } \text{arr})\ B(\text{item})$	<code>for(i=0;i<e.len;i++)f(e[i]);</code> <code>→对于(e中的每一项) f(项);</code>
结构折叠	格式折叠	令 W 为非必要的空白字符: 对于任意字符串 s , $T(s) \rightarrow s'$, 其中 $\forall w \in W, w \notin s'$ 除非需要分隔词元。	<code>如果 (x > 0) { y = 1; } → 如果(x>0)y=1;</code>

步骤一：规则检索。此步骤通过训练好的分类器将查询匹配到相关簇，并在选定的簇内进行代码搜索。

首先，我们从查询中提取关键词，同时过滤掉诸如“if”和“the”等语义价值较低的停用词。我们仅使用BERT对最高频的关键词进行编码 [9]，而不是对整个查询文本应用标准的嵌入方法。这一设计的动机在于，我们观察到非专家用户提出的自然语言查询通常包含冗余或描述性的措辞，这可能会引入语义噪声并降低聚类准确率。相比之下，TAP规则的核心意图通常通过一组集中的名词和动词来表达。通过专注于这些前K个词汇特征，我们获得了一个更稳健且以意图为中心的表示，这提高了聚类预测以及后续代码检索的精确率。生成的嵌入被输入到训练好的分类器中，以预测对应的簇。为了减轻仅依赖最高概率预测可能导致的误分类风险，我们保留按概率排序的前三个聚类，以减少最高排名聚类中偶尔出现的错误所带来的影响。

然后，我们在选定的聚类中进行代码检索。用户查询和选定聚类中包含的代码都使用大语言模型（例如，CodeBERT[11]）进行编码。我们计算查询嵌入与每个嵌入之间的余弦相似度。相似度分数最高的规则被选为代表示例，因为它最准确地反映了底层的控制逻辑。

步骤二：上下文检索。此步骤分析用户的查询以推断其目标及涉及的实体，然后从家庭上下文中检索相应设备的状态和功能。

首先，我们使用大语言模型作为目标和实体识别器。给定用户查询，我们提示大语言模型提取两个关键要素：1) 主要目标（例如，“让房间更凉爽”、“节省能源”、“增强安全”），以及 2) 涉及的实体类型或特定实体的列表（例如，[空调、客厅], [温度传感器、客厅], [灯、卧室]）。这使得 HomeGenii 即使从省略了具体设备名称的模糊描述中也能理解用户的意图。随后，HomeGenii 对家庭上下文数据库执行语义搜索。该数据库包含所有可用设备及其类型和功能的实时清单。上一步推断出的目标和实体列表被用作查询。我们计算此查询与家庭上下文中实体描述之间的语义相似度。然后，系统对最相关的实体进行排序和检索，这些实体能够满足用户陈述的目标。

5.4 压缩增强器

压缩增强器会对上述检索到的TAP规则进行压缩并执行增强步骤。为了在保留核心内容的同时减少令牌开销，我们引入了三项设计原则来指导压缩过程：

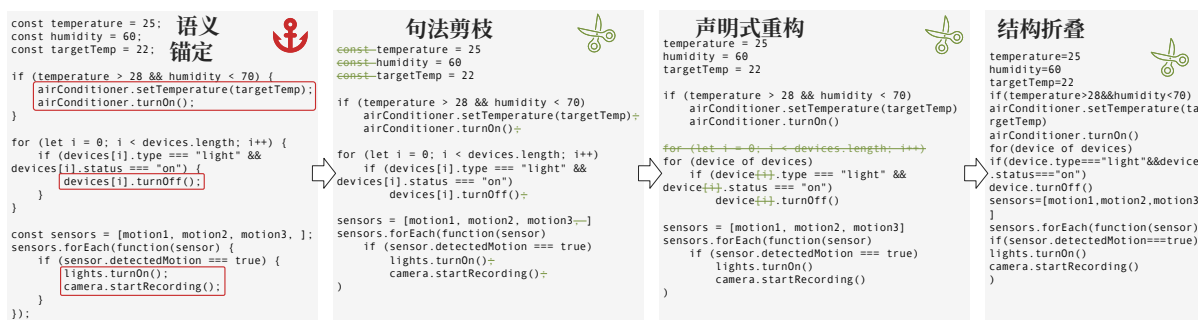


图6. 压缩IFTTT过滤器代码的示例。

- **原则一：语义不变性。** 那些定义了与环境交互的基本行为的元素，在压缩过程中必须保持其语义不变。
- **原则二：句法消除。** 高级编程语言中的语法糖对于大语言模型而言是冗余的，应当被移除。
- **原则三：声明式表达。** 通过将过程式表示转换为声明式表示来精简结构模式，以提升规则密度。

基于以上原则，我们首先识别并提取关键的语义行为（**步骤一**），然后将原始TAP转换为更紧凑的中间表示（**步骤二**），最后通过大语言模型生成可执行规则（**步骤三**）。

步骤一：语义锚定。 此步骤遍历检索到的规则的抽象语法树，以识别并保护“语义锚点”。这些锚点是决定规则行为的不可变节点，包括标识符和API调用签名。标识符是维护规则内部状态和逻辑的变量与功能名称。API调用签名是最关键的锚点，因为它们代表了规则与智能家庭环境的交互。这些调用天然遵循远程过程调用模式（例如，`Weather.currentWeather[0].CurrentCondition`）；RPC格式明确、紧凑，且能被目标平台和大语言模型直接解释，使其成为语义保存和词元效率的理想表示。如图6的语义锚定阶段所示，这些锚点被识别出来，并构成所有后续转换的不可变核心。

步骤二：多遍微型化。 此步骤对源TAP规则应用一系列转换遍次，逐步将其压缩为微型化的中间表示。该过程详情见表3，可视化展示见图6，包含按顺序应用的三大类转换规则。首先，句法剪枝剥离对大语言模型而言多余的语法，包括声明关键字（如 `const` 和 `let`）、非必要的标点（如结尾分号）以及部分块括号，这在向“句法剪枝”列的转换中可见。其次，声明式重构通过用声明式等价物替换冗长的过程式代码来增强表达力。此示例展示了一个C风格的 `for` 循环被重构为更清晰的 `for...of` 结构，以及一个错误的数组元素被纠正并最终移除。最后，结构折叠通过消除所有非必要的空白（包括换行和缩进）来最大化词元效率，从而将整个规则压缩为密集的单行格式，如最右侧“结构折叠”列所示。请注意，本文为演示而使用JavaScript，因其丰富的句法特性提供了更大的压缩潜力，但底层的压缩原则设计为可适配。对于采用不同语言的目标平台，例如Home Assistant中的Python脚本，可以相应调整微型化规则以保持兼容性和有效性。

HomeGenii: 我已为您设置了一条新规则。以下是它将执行的操作:

- **高层意图 (您的目标)**
在空气净化器运行后, 我希望你检查空气质量。如果仍然很差, 向我发送一个关键警报以检查过滤器。如果只是中等, 则打开风扇运行一段时间以帮助空气循环。否则, 只需告知我空气质量良好即可。
- **中层计划 (系统将如何执行)**
触发器: 当[空气净化器]开启时。
动作序列:
 - (1) 从[客厅空气质量指数传感器]获取当前的空气质量指数 (AQI) 值。
 - (2) 然后, 执行以下检查:
 - 如果空气质量指数值高于100, 则向[用户]的手机发送一条高优先级通知, 内容为消息: “空气质量差! 请检查窗户和净化器过滤器。”
 - 否则, 如果空气质量指数值高于50, 则开启[吊扇] 15分钟。
 - 否则 (如果空气质量指数为50或以下), 则发送通知: “空气质量现在极佳。”
- **底层实现 (最终代码)**
(详细的TAP代码, 此处因篇幅限制省略)

这看起来对吗?

看起来不错!

做出更改

取消

图7. 一个用户参与循环验证的示例。

步骤三：大语言模型提示与最终规则生成。最后一步将检索到的信息组装成一个增强提示，该提示由四个按固定顺序排列的组件构成：

- (1) 指令前导码：定义大语言模型角色和任务的元指令。它明确指出目标是为目标平台生成TAP代码，并且需求和相关信息将被提供。所提供的信息经过压缩但在语义上忠实，应作为控制逻辑的重要参考。
- (2) 目标平台模式：一个指定目标平台所需输出格式的模板。
- (3) 用户的原始查询：用户未修改的自然语言请求。这为生成任务提供了主要目标和上下文。
- (4) 增强信息：由我们的多阶段小型化过程产生的中间表示。此外，它还包括环境上下文。

最后，基于这个结构化提示，大语言模型为目标平台合成最终的TAP规则。

5.5 规则检查器

在部署大语言模型生成的规则之前，规则检查器会执行一个两阶段验证过程：(1) 内部自动化验证，以及 (2) 可选的用户参与循环验证以实现意图对齐。

步骤一：自动化验证。此步骤会根据目标平台的已知约束，自动验证生成的规则。其目标是检测并过滤掉因大语言模型幻觉而产生的技术缺陷规则和错误。该分析包含三项主要检查。

首先，根据平台的正式模式解析规则。对于 Home Assistant，这意味着验证 YAML 结构，确保必需的键（例如，触发器、动作）存在，数据类型正确，且语法有效。接下来，我们维护一份用户智能家居环境的清单，列出所有可用的实体_ID 和可调用服务。然后，将规则与此清单进行交叉引用，以标记任何对不存在的设备（例如，light.nonexistent_lamp）或无效服务（例如，light.set_color_name 而非 light.turn_on）的引用。最后，我们检查服务调用参数的有效性。例如，如果一个简单的可调光灯泡被错误地调用了 color_temp 参数，我们会将其标记为不匹配，以防止运行时错误。

自动化验证的能力与局限。上述检查主要是静态和语法层面的，旨在确保平台合规性、设备兼容性，以及代码格式良好且可执行。然而，它们并不评估更高层次的逻辑正确性或意图对齐，例如条件分支是否准确反映了用户描述的场景，或者规则是否可能导致意外的副作用（例如无限循环）。这种固有的局限性凸显了后续用户参与循环验证阶段的必要性，该阶段将焦点从语法正确性转向语义适当性。

步骤二：用户参与循环验证。此步骤的焦点从正确性转向适当性。为促进这一点，我们首先将已验证的规则翻译成对其行为的简洁、人类可读的摘要。受SH-RDL启发，[5]，我们利用大语言模型将生成的规则转化为三个层次化的抽象级别：高层意图、中层计划和底层实现。高层意图用自然语言总结规则的目的，中层计划以结构化、人类可读的格式概述技术逻辑，而底层实现则包含实际可执行的规则。随后，用户将获得修改规则的选项。图7 是一个示例。

用户可通过三种交互选项对生成结果进行控制。选择看起来不错！将授权立即部署该规则。如果用户发现差异并选择做出更改，则可以提供自然语言反馈（例如，“将触发时间改为晚上10点”或“保持床头灯开启”）。此反馈随后被视为一个精炼查询，并与原始上下文结合，反馈至 HomeGenii 的流程（检索、压缩、生成和验证）中，从而启动一个新的迭代优化循环。选择取消则会完全中止该过程。

6 评估

在本节中，我们首先描述实验设置。然后评估系统在规则生成方面的整体性能。接着，我们分析其五个核心组件的有效性。随后通过一个案例研究来展示系统在生成复杂自动化规则时的运作方式。最后，我们进行了一项用户研究，以考察日常用户在与 HomeGenii 交互时的实际体验和反馈。

6.1 实验设置

6.1.1 实现。我们将HomeGenii部署在一台边缘设备和一台私有云服务器上，并与一个商业LLM服务集成。

边缘。一台树莓派5作为边缘的本地控制器。该节点负责：(1) 收集用户查询及家庭环境上下文并转发至私有云，以及 (2) 托管目标自动化平台（例如Home Assistant）以执行最终验证过的TAP规则。

私有云。该层级托管于我们自管理的服务器上，配备AMD EPYC 7V12（64核）CPU、120 GB DDR4内存和NVIDIA RTX 4090 GPU。其职责包括：(1) 托管TAP规则的外部规则库；(2) 运行轻量级 CodeT5 [46] 编码器以进行代码嵌入和语义搜索操作；(3) 构建增强提示；(4) 管理与公共云层级的通信，发送提示并接收初始的大语言模型生成的规则；(5) 执行自动化验证并协调自校正循环。

表4. 不同家庭配置的设备。

Home	气候/舒适度	照明/遮阳	电器/固定装置	安防/存在感应
家庭1	–	飞利浦Hue白色灯泡; Yeelight LED 1S; Kasa智能灯泡	Hamilton Beach Coffee 制造商; Kasa智能插座; WeMo智能插座	小米运动传感器; WeMo 运动传感器
Home 2	Ecobee智能 恒温器; Sensi 湿度传感器; 小米 空调控制器; Mi 温湿度 传感器	飞利浦Hue白色灯泡; Kasa 智能灯泡; 飞利浦Hue智能网关; 小米智能电脑显示器 灯带1S	Keurig K-Elite 咖啡 ; WeMo智能插座; 涂鸦智能开关	Ring 门铃; 米家智能门锁 /窗户传感器 2; 智能- Things Motion
Home 3	Eight Sleep Pod; Room AC; 小米空调 控制器; SmartThings 多传感器; 小米温湿度传感器; Sensi 湿度传感器 湿度传感器	路创 Serena 智能窗帘; Hue智能灯泡; 小米窗帘控制器; 小米智能计算机 显示器灯带1S; Yeelight LED 1S; Kasa智能灯泡	Broan智能抽油烟机; 小米 扫地机器人; SmartThings 智能插座; 涂鸦智能开关; WeMo智能插座; Hamilton Beach 咖啡机	Kasa智能摄像头; 智能- Things 运动; 米家智能门锁 /窗户传感器 2; WeMo 运动传感器; 小米运动 传感器
家庭 4	Room ACs; 除湿器; 加湿器; Ecobee智能 恒温器; Mi 温湿度 传感器	Hue智能灯泡; 路创智能窗帘; 飞利浦Hue智能网关; 小米窗帘 控制器; 小米智能电 脑显示器灯带1S	Rachio 3智能洒水器; 厨房 吊扇; 抽油烟机; WeMo智能插座; 米家 智能宠物喂食器; Keurig K-Elite咖啡机; Broan 智能抽油烟机	Furbo宠物摄像机; Wyze 智能摄像头; Ring Doorbell; 小米智能网关; SmartThings Hub; Kasa Smart 智能摄像头; 叮咚 智能视频门铃; 米家智能门锁 /窗户传感器 2; Smart- Things 多传感器

大语言模型 服务。该组件是商用大语言模型服务，专门用于生成TAP规则。我们选择了三种先进的商用大语言模型：ChatGPT-5、Gemini-2.5 Pro和DeepSeek-R1。

6.1.2 家庭设置。我们定义了四种复杂度递增的家庭配置（见表4）：家庭1（基础型）是一个最小信息设置；家庭2（增强型）增加了用于情境感知规则的传感器；家庭3（多品牌型）是一个设备密集、多品牌的环境，用于评估互操作性；家庭4（安全型）包含以安全为核心的设备，用于复杂的安全自动化。每种配置都在一个 Home Assistant 实例中实现，其中所有设备都被分配到你对应的功能区域。

6.1.3 数据集。我们的数据集包含规则库和用户查询。规则库。我们策划了一个包含101条增强型TAP规则的专家规则库，其构建遵循第5.1节概述的方法论。构建工作由两位在智能家居系统和大语言模型应用方面具有专业知识的博士研究生承担。结构化流程包括：(1) 定义四层分层分类法，(2) 将意图分解为触发-行为对，(3) 将其映射到设计模式以合成实例，以及 (4) 将实例抽象为参数化模板，总共大约需要5人时。示例自动化规则如表5所示。

用户查询。我们使用ChatGPT合成一套用户查询，其中一些查询有意设计得模糊不清，这与第4.2节中的描述一致。为了防止系统在评估过程中依赖肤浅的关键词匹配，我们还引入了这些查询的词汇和句法改写。

6.1.4 基线方法。我们通过将我们基于RAG的HomeGenii 与三种基线方法进行比较，来评估其规则生成性能。第一种基线（标记为原始方法）直接提示大语言模型根据用户查询生成规则，不提供任何外部信息或上下文。第二种基线（标记为思维链方法）采用思维链 [47]，我们向大语言模型提供几个增强型TAP规则的示例

表5. 来自层次分类法的示例自动化规则。

自动化名称	层级	描述
入侵警报	1	当安全系统布防时检测到运动，立即打开所有灯具，并触发警报器，同时向房主和安保服务发送紧急通知。
漏水预防	1	监控地下室和家电附近的水传感器；一旦检测到湿气，自动关闭主水阀并提醒房主，以防止水淹损害。
自适应恒温器	2	根据占用模式、天气预报和电价调整暖通空调设置，以在保持舒适度的同时最小化能耗。
智能灌溉	2	根据土壤湿度读数、天气条件和植物种类为花园浇水，并在降雨期间跳过浇水周期以节约用水。
睡眠环境 优化器	3	在夜间逐步调整卧室温度、照明和白噪音，以促进深度睡眠周期和自然醒。
空气质量管理器	3	监测PM2.5、二氧化碳和挥发性有机化合物水平；当污染物超过健康阈值时，自动启动空气净化器并调节通风。当污染物超过健康阈值时。
晨间例程	4	创建个性化的唤醒序列，包含渐进式照明、偏好的新闻简报和咖啡机器激活，基于个人日程和偏好。
娱乐场景	4	一键“电影之夜”模式，可调暗灯具、关闭窗帘、调节温度并配置音视频设备，以获得最佳观看体验。

，以引导其逐步推理如何根据用户的查询生成合适的规则。第三种基线（标记为长上下文方法）将规则示例作为长上下文提示提供给大语言模型，而不是使用RAG方法向模型提供信息。我们还选择了两个最先进的、基于大语言模型的系统作为基线：Sasha [26] 和 ChatIoT [13]。根据其原始设计，ChatIoT的输入包括：(1) 用户请求，(2) 可能涉及的实体的属性列表，以及 (3) 包含这些属性详细信息的上下文。Sasha的输入包括：(1) 任务指令，(2) 描述房间、设备、传感器和设置的JSON“家庭模板”，以及 (3) 用户的命令。然而，它们没有被提供增强型TAP规则的示例，因为它们的架构本身不支持此类结构化代码示例。直接进行开箱即用的比较是不可行的，因为这两个基线都不是为生成增强型TAP脚本而设计的。为了确保一个公平且聚焦的比较，以将生成复杂控制逻辑的核心能力与仅仅熟悉特定平台语法区分开来，我们对这两个系统进行了调整。我们保留了它们原始的提示工程和推理流程，但将其最终输出格式化阶段替换为与HomeGenii所使用的相同的精确指令。这最后的提示明确定义了目标Home Assistant模式、所需的数据结构和应用程序接口约定，确保所有系统都针对相同的技术规范。

我们通过具体比较进一步评估了HomeGenii中关键技术的有效性。对于语义聚类器，我们将其性能与未聚类的基线以及经典的K-means算法进行比较，后者基于从CodeT5获得的TAP规则嵌入进行操作。信息检索器则与一个基线进行比较，该基线通过使用CodeT5编码查询并计算嵌入之间的余弦相似度来执行上下文检索。最后，为了评估压缩增强器的有效性，我们将其与两个基线进行比较：Pyminifier [29]，（一个源级Python代码压缩器，通过移除冗余语法来减少令牌使用量，同时保留功能）和LLM-Sum@N（它使用大语言模型将检索到的规则总结为大约N个词元的伪代码）。

6.1.5 评估指标。我们采用三个指标来评估规则生成性能：准确率、CodeBLEU和令牌消耗。使用准确率和CodeBLEU与我们形成性研究的设计保持一致（参见第3.1节）。此外，我们引入令牌消耗作为量化经济开销的指标。

为评估规则库质量，我们首先通过沿三个维度分析数据集来衡量不同分类意图之间的正交性：触发器相似度、动作相似度以及触发器-动作（小程序）

表6. 与相关工作相比，在不同大语言模型下增强型触发-动作编程生成的性能。

大语言模型: ChatGPT-5												
方法	家庭1			Home 2			Home 3			家庭 4		
	Acc.	CodeBLEU	词元	Acc.	CodeBLEU	词元	Acc.	CodeBLEU	词元	Acc.	CodeBLEU	词元
原始	14.27	25.33	918.2	15.18	23.88	932.8	14.77	25.21	930.8	15.21	24.76	928.6
CoT	22.40	33.40	3830.0	18.80	32.20	3811.2	21.20	34.20	4812.6	20.40	31.20	4857.2
LC	45.60	42.73	59421.4	44.20	42.49	60189.8	43.60	41.15	61342.2	45.60	42.01	61566.0
Sasha	22.97	33.14	3780.0	19.60	32.60	3891.2	20.20	33.59	4712.6	20.79	31.76	4887.2
ChatIoT	43.56	39.51	4876.4	42.18	39.27	5033.2	41.58	37.92	5985.4	43.56	38.78	6114.8
HomeGenii	81.19	58.58	2418.2	80.59	59.28	2678.6	78.22	59.50	3306.2	79.80	59.83	3416.2
大语言模型: Gemini-2.5 Pro												
方法	家庭1			Home 2			Home 3			家庭 4		
	Acc.	CodeBLEU	词元	Acc.	CodeBLEU	词元	Acc.	CodeBLEU	词元	Acc.	CodeBLEU	词元
原始	13.65	23.73	1027.4	14.55	22.28	1084.2	14.27	23.61	1090.2	13.56	23.11	1093.8
CoT	22.60	34.20	3862.6	22.80	33.20	3889.2	24.60	32.80	5206.8	22.40	31.60	5191.4
LC	53.80	45.62	59715.2	54.20	45.99	60432.8	54.60	44.12	61208.4	55.20	44.78	61891.2
Sasha	22.18	33.94	3812.6	23.56	33.50	3969.2	23.56	32.17	5106.8	22.77	32.19	5221.4
ChatIoT	51.88	42.38	4882.8	52.28	42.77	5011.6	52.67	40.88	5917.6	53.27	41.54	6159.8
HomeGenii	84.75	59.81	2459.4	84.36	60.57	2571.8	82.18	58.74	3229.4	83.37	58.20	3386.4
大语言模型: DeepSeek-R1												
方法	家庭1			Home 2			Home 3			家庭 4		
	Acc.	CodeBLEU	词元	Acc.	CodeBLEU	词元	Acc.	CodeBLEU	词元	Acc.	CodeBLEU	词元
原始	13.98	31.95	908.8	14.18	32.47	913.2	14.37	29.82	925.4	14.80	29.36	921.4
CoT	19.80	34.60	3953.4	18.00	33.40	4045.8	19.80	36.20	4978.6	18.00	32.80	4892.2
LC	42.40	42.11	59812.6	39.40	43.91	60544.4	38.60	41.70	61123.8	41.20	43.15	61388.6
Sasha	19.21	34.22	3903.4	18.61	33.80	4125.8	18.81	35.67	4878.6	18.22	33.29	4922.2
ChatIoT	40.40	38.88	4929.2	37.43	40.68	5162.4	36.63	38.47	5973.8	39.21	39.92	6046.8
HomeGenii	80.20	53.79	2571.2	78.22	52.63	2711.6	77.23	54.24	3576.2	80.20	51.77	3622.8

相似度。层级之间的相似度使用杰卡德距离计算。此外，我们分析IFTTT小程序在层级1-4中的分布，以评估分层分类法的覆盖范围。

规则检索性能使用成功率@K（SR@K）和平均倒数排名（MRR）进行评估。SR@K衡量的是在检索结果的前K项中至少出现一个正确结果的查询比例。MRR计算的是第一个正确结果的平均倒数排名。

对于上下文检索性能，我们采用召回率、精确率和F1分数。精确率衡量检索到的上下文实体中相关实体的比例。召回率衡量所有相关实体中被成功检索到的比例。F1分数提供了精确率和召回率的调和平均值。

压缩性能通过令牌压缩率（TCR = 1 - 压缩后词元数/压缩前词元数）来量化令牌减少量，并通过准确率下降（AD = 压缩前准确率 - 压缩后准确率）来评估潜在的质量下降。

6.2 规则生成性能

本实验评估了HomeGenii 在生成增强型TAP规则方面的性能。我们在四个家庭设置（Home1-Home4）中执行评估。将我们数据集中的用户查询作为输入，提供给HomeGenii 和各基线方法，分别在每个大语言模型：ChatGPT-5、Gemini-2.5 Pro和DeepSeek-R1下进行。表6中呈现的结果是针对每个特定条件在所有查询上计算出的平均值。

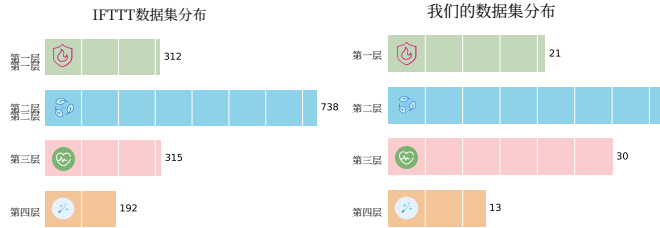


图8. IFTTT自动化规则的分布。

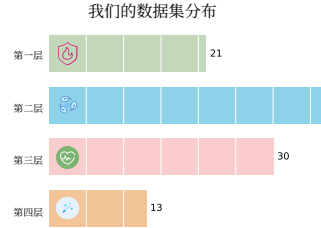


图9. 我们数据集中自动化规则的分布。

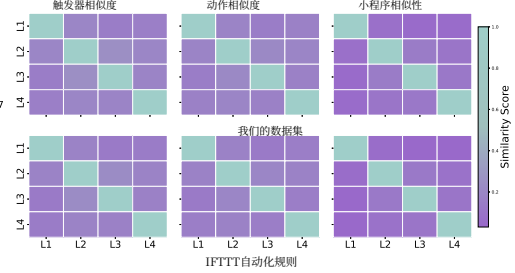


图10. IFTTT与我们数据集的相似度矩阵。

准确率。在准确率指标上，HomeGenii 在多个家庭中获得了约80%的平均准确率，显著优于原始方法（15%）、思维链（21%）、LC（46%）、Sasha（21%）和ChatIoT（43%）。在基线方法中，LC在准确率方面表现最佳。这表明，即使不使用RAG，提供丰富信息也能改善生成结果，这与第3节中的发现一致。然而，由于未使用RAG，LC的性能并不令人满意，因为过多的丰富信息可能会干扰大语言模型的生成。思维链的表现与Sasha接近，因为Sasha采用了一种与思维链提示策略非常相似的方法。实现了多智能体系统的ChatIoT取得了相对较好的结果。尽管如此，这些基于提示的策略由于缺乏外部知识，仍然难以生成增强型TAP所需的复杂控制逻辑、API使用模式和数据结构。

CodeBLEU。CodeBLEU结果为生成的规则的结构质量提供了补充性见解。HomeGenii始终获得最高分数（例如，在ChatGPT-5下为59%，而基线方法分别为15%、ChatIoT为38%、Sasha为32%），这一趋势在所有大语言模型中保持稳定。这表明其输出在结构上与专家编写的示例更为一致。虽然一个功能完善的增强型触发-动作编程脚本有时可能偏离特定的真实实现，从而导致较低的CodeBLEU分数，但HomeGenii的高准确率与高CodeBLEU分数之间的一致相关性证实了其检索增强方法能有效引导大语言模型生成既正确又结构良好的代码。

令牌消耗。HomeGenii消耗的令牌也比基线方法少。尽管ChatIoT的提示设计包含了压缩器，但我们的测试表明，其压缩技术即使在结合了更现代的句子BERT模型后，仍会对增强型触发-动作编程的生成质量产生负面影响。因此，报告中ChatIoT的令牌数量反映了其未压缩操作的情况。LC在令牌消耗方面表现出最明显的劣势，因为将全部规则作为每个提示的输入成本过高。即便如此，LC仅达到约46%的准确率，远低于HomeGenii。

6.3 规则库构建的有效性

为验证规则库构建方法的有效性，我们首先评估层次分类法是否能全面覆盖自动化规则。我们从IFTTT网站共收集了1557个推荐小程序。如图8所示，第一层至第四层分别包含312、738、315和192个小程序，与层次结构完全对应。随后，我们计算了IFTTT数据集中不同层级之间的相似度。如图10所示的结果表明，各层级间的相似度分数较低，大多数值低于0.2。这证明了所提出的层次分类法各层级间具有较高的正交性。此外，我们还分析了所构建数据集的分布情况（如图9所示），并计算了相似度矩阵。

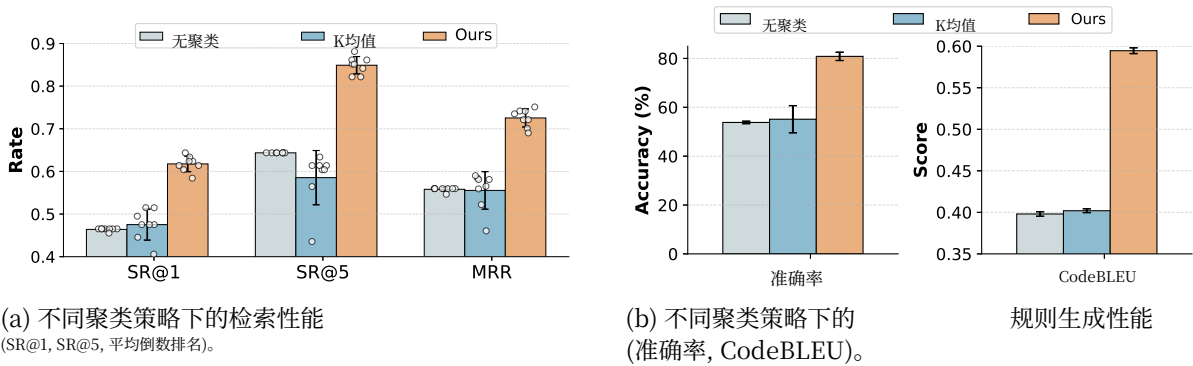


图11. 聚类策略对检索和生成性能的影响。

表7. 基于大语言模型和查询嵌入的上下文检索方法的性能比较。

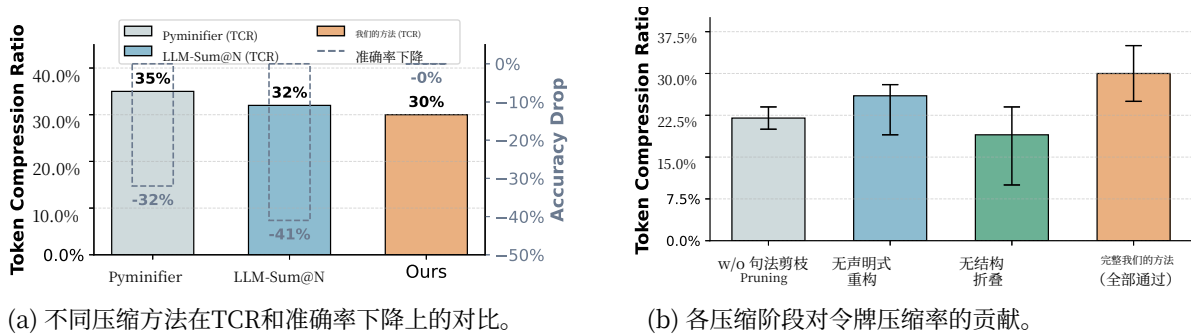
方法	Home	上下文检索性能			规则生成性能	
		召回率	精确率	F1	准确率	词元
基于大语言模型的检索	家庭1	98.88	81.67	89.63	81.19	2418.20
	Home 2	98.73	87.68	92.80	80.59	2678.60
	Home 3	96.24	84.50	91.33	78.22	3306.20
	家庭 4	98.57	87.11	92.32	79.80	3416.20
查询嵌入	家庭1	25.33	61.87	36.14	17.58	2307.20
	Home 2	27.67	60.42	36.79	18.79	2594.80
	Home 3	22.28	58.91	32.47	17.22	3086.20
	家庭 4	28.72	59.33	36.08	17.81	3280.20

6.4 语义聚类的影响

我们从两个关键角度评估语义聚类方法的有效性：其对规则检索的影响，以及其对最终规则生成质量的后续影响。

聚类对规则检索的影响。如图11a所示，我们的语义聚类方法实现了62%的SR@1，优于基于嵌入的检索基线（无聚类，46%）和K均值聚类（47%）。这一结果证实，通过语义相似度对规则进行分组，显著增强了系统从规则库中检索功能相关规则的能力，尤其是在处理模糊或未充分指定的用户查询时。

聚类对规则生成的影响。高质量检索的益处延伸至端到端的生成性能，如图 11b所示。在相同的输入条件下，我们的语义聚类取得了%的准确率，而K均值为55%，无聚类为54%。在CodeBLEU指标上，我们的方法达到约0.59，优于K均值（0.40）和无聚类（0.39）。



(a) 不同压缩方法在TCR和准确率下降上的对比。

(b) 各压缩阶段对令牌压缩率的贡献。

图12. 压缩性能评估。(a) 各方法在令牌压缩率与准确率下降之间的权衡。(b) 消融研究分析不同压缩阶段对总体令牌压缩率的贡献。

6.5 上下文检索性能

本实验评估了我们提出的基于LLM的上下文检索机制与查询嵌入基线在四种家庭配置下的性能。本研究使用的生成模型是Gemini-2.5 Pro。如表7所示，基于LLM的检索方法在所有家庭的召回率、精确率和F1分数方面均持续且显著优于查询嵌入方法。例如，在家庭1中，基于LLM的方法取得了89.63的F1分数，远超查询嵌入方法的36.14。更优的检索质量直接促成了更好的规则生成。基于LLM的方法凭借其更优的检索质量，直接提升了规则生成效果，实现了约80%的准确率，而查询嵌入方法仅约18%。由此带来的性能提升证明了令牌使用量的适度增加是合理的。

6.6 令牌压缩的有效性

本节通过对比基线技术及其构成阶段的消融分析，评估我们的令牌压缩方法。

压缩对规则生成的影响。我们将压缩增强器与基线方法应用于相同的检索到的TAP规则，并在相同的大语言模型设置下生成最终规则来进行评估。每种方法在控制检索质量和解码参数的情况下运行十次试验。图12a的结果显示，我们的方法实现了30%的令牌压缩率且无准确率损失（准确率下降 $\approx 0\%$ ），优于Terser（令牌压缩率=35%，准确率下降=32%）和LLM-Sum@N（令牌压缩率=32%，准确率下降=41%）。因此，我们的方法在完全保持生成质量的同时，实现了显著的令牌节省。

消融研究。我们通过系统性地消融三个变体来分析每个压缩阶段的贡献，每个变体分别排除以下组件之一：句法剪枝、声明式重构或结构折叠，并将它们与完整模型进行比较。如图12b所示，移除任一阶段都会降低TCR。省略结构折叠导致降幅最大，TCR降至约19%，这证实了其核心作用。没有句法剪枝，TCR降至22%，突显了其在抽象冗余句法方面的贡献。移除声明式重构导致TCR约为26%，表明其影响中等但对样本敏感。

6.7 HomeGenii实践：案例研究

为了展示HomeGenii在现实场景中的实际应用，我们开发了一个基于Web的用户界面，并在一个模拟家庭环境中进行了案例研究。本节将说明最终用户如何与HomeGenii交互以创建复杂的自动化规则。

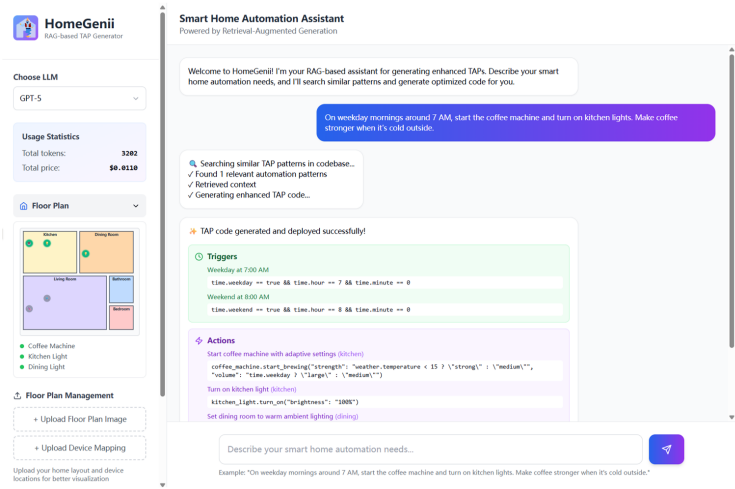


图 13. HomeGenii 演示系统的用户界面。

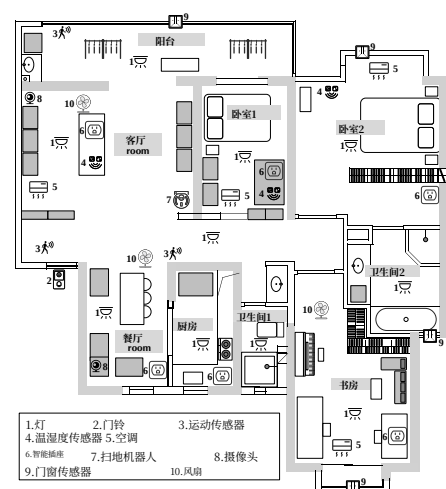


图 14. 案例研究中的智能家居布局。

6.7.1 用户界面。我们使用Vue 3作为前端、Spring Boot作为后端实现了一个Web应用程序，为用户与HomeGenii交互提供了一个直观的界面。如图13所示，界面由几个关键组件组成。左侧面板显示家庭结构，其中包含按房间层次和功能设备组织的设备分组。用户可以通过“平面图管理”部分更新平面图图像和设备映射，以增强空间上下文的可视化。主要的交互区域是一个对话式界面，用户可以用自然语言描述其自动化需求。生成成功后，界面会以结构化格式呈现最终的增强型TAP代码，并使用平台兼容的语法清晰地分隔触发器和动作。

6.7.2 家庭布局。图14展示了我们案例研究中使用的家庭设备布局，这是一个包含客厅、餐厅、阳台、卧室、浴室、厨房和书房的多房间住宅。十种不同类型的设备被布置在关键区域和与常见家庭自动化场景相关的潜在交互点。

6.7.3 示例案例：自动晚间关闭。本案例展示了 HomeGenii 如何通过其集成流程，将用户查询“晚上11点关闭所有灯，但保持安防设备开启”转化为经过验证、可执行的自动化。图15详述的端到端过程，展示了系统检索相关规则和上下文、压缩提示以及生成可执行的增强型TAP规则的能力。

信息 检索。给定自然语言输入，HomeGenii 检索到一个定时关灯自动化的增强型TAP规则以及相应的上下文，其中包括灯具和安全传感器。我们注意到，检索到的规则原始触发时间为 22:00:00，这与用户期望的 23:00:00 不同。检索 TAP 规则耗时 1.09 秒且不消耗词元，因为它完全依赖于本地语义搜索。相比之下，检索上下文设备信息需要 2.23 秒并使用 918 个令牌。

规则 压缩。检索到的规则被压缩成一个紧凑提示。此过程将原始内容缩减为一种最小信息表示，同时保留关键的语义锚点，例如触发时间、设备类别和关闭/保持逻辑。作为一种基于规则的操作，压缩引入的延迟可忽略不计，且需要零令牌。

规则 生成。使用压缩后的提示，大语言模型生成一个语法上合规的 TAP 规则。生成的输出正确地将触发器时间设置为 23:00:00，并包含一个 Python 脚本，该脚本清晰地分离

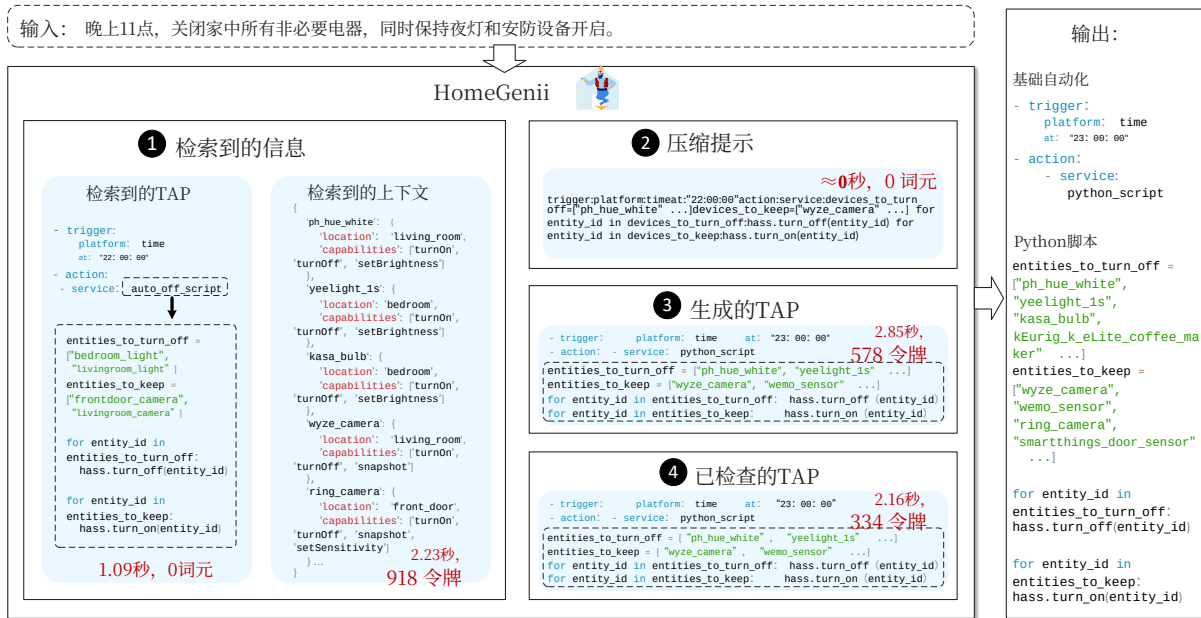


图 15. 案例：自动化晚间关闭。

通用照明实体与安防设备。此生成步骤消耗了578个词元，反映了将结构化上下文转换为可执行的Home Assistant代码的成本。

规则验证。生成的规则会通过一个基于规则的流程自动检查平台合规性，该流程不产生词元成本且延迟极低。随后，规则会被转换为多个抽象层级以供用户呈现。此抽象步骤消耗334个令牌，并在2.16秒内完成。在收到用户确认后，系统将输出最终的增强型触发-动作编程规则。

6.8 用户研究

6.8.1 参与者。我们招募了15名参与者（P1–P15；八名男性，七名女性；参见表8）。他们的年龄从18岁到64岁不等，职业和教育背景多样。大多数参与者（n=9）年龄在25–44岁之间，且多数（n=10）拥有学士学位或更高学历。参与者的智能家居使用经验从无到超过两年不等，相应的使用频率也从从不使用到每日使用。

6.8.2 基线。参与者同时使用了HomeGenii和基线系统。基线系统的配置方式是移除HomeGenii中用于检索规则示例的组件，仅提供家庭上下文。此配置更符合ChatIoT或Sasha等先进系统的设置。此外，基线系统中排除了规则检查器模块。

6.8.3 流程。首先，我们向参与者提供了附录A.1节中描述的教程，以学习如何使用HomeGenii（基线界面与此类似）。完成此教程大约需要十分钟。在熟悉系统后，每位参与者被要求同时使用HomeGenii和基线系统来提交10个用于生成TAP规则的查询。这些查询涵盖了不同复杂度的场景。参与者根据他们的理解，将相应需求输入到HomeGenii和基线系统中，并被要求在过程中进行出声思维，讲述他们的体验。最后，在完成

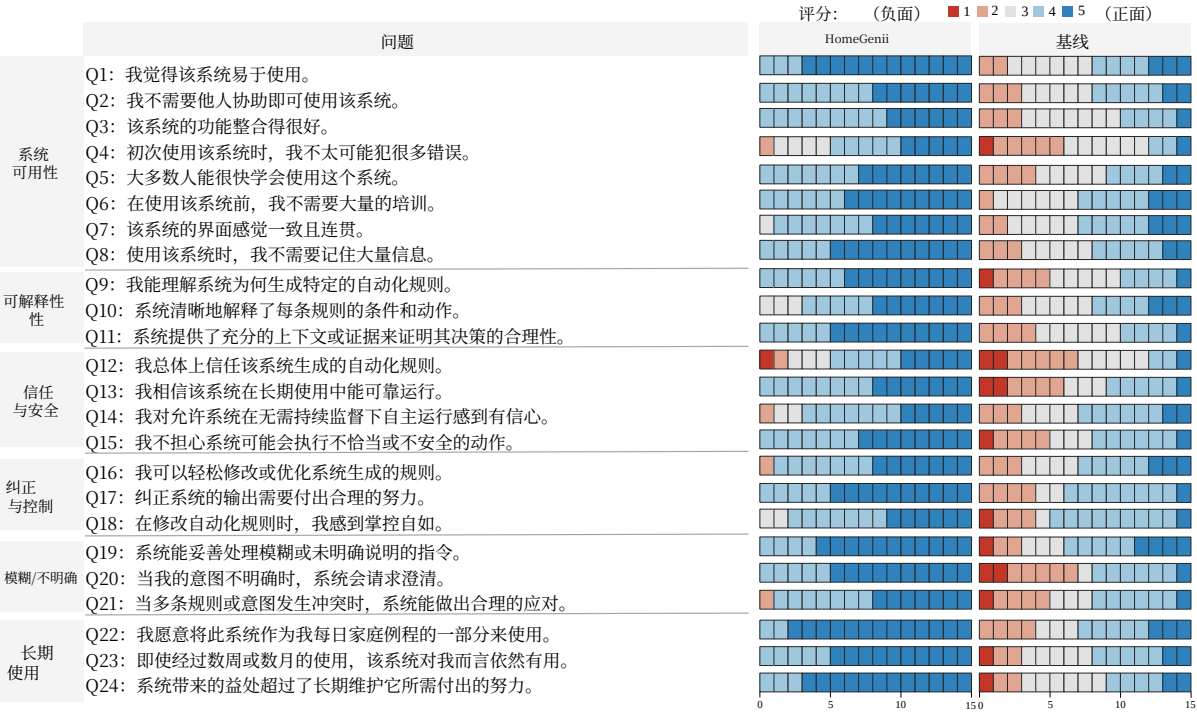


图16. 用户对HomeGenii与基线的问卷评分。

任务后, 参与者填写了一份问卷 (如图16所示)。每次会话持续约2小时, 参与者因其付出的时间获得了40美元的报酬。

6.8.4 问卷设计。 我们的问卷围绕家庭自动化系统的六个维度构建: 系统可用性、可解释性与透明度、信任与安全、用户纠正与控制、模糊/不明确意图处理以及长期使用与采纳。每个维度均采用标准的5点李克特量表进行测量[23], 其中包含专门设计的具体条目, 旨在探究系统的实际可用性以及用户的心理接受度。例如, 关于信任与安全的条目评估用户对自主操作的信心, 而关于模糊意图的问题则评估系统在现实场景中处理指令不完整或相互冲突时的稳健性。

6.8.5 结果分析。 为了定量评估HomeGenii与基线系统在用户评分上的差异, 我们对全部24个问卷条目进行了Wilcoxon符号秩检验[48]。结果总结于图16中, 揭示了用户在多个维度上对HomeGenii的一致偏好。

可用性。 参与者认为HomeGenii更易于使用和学习 (Q1、Q2、Q5、Q6: $p < .01$)。通过检索专家示例, HomeGenii提供的输出感觉逻辑清晰且可预测, 减少了用户以类编程精度表述请求的需求。此设计也使系统感觉更集成、更一致 (Q3、Q7: $p < .05$), 并降低了使用所需的脑力负担 (Q8: $p < .01$)。

可解释性。 HomeGenii 在此方面得分显著更高 (Q9, Q11: $p < .001$)。其三层解释 (意图、计划、代码) 有效地揭示了生成的规则。用户反馈他们能够理解规则创建的原因并追溯其逻辑, 这使得系统的决策显得合理而非不透明。

信任与长期使用。虽然两个系统在基本操作上都获得了信任（Q12, Q14: n.s.），但HomeGenii在自主、长期使用方面激发了更大的信心（Q13, Q15, Q23: $p < .01$ ）。这种经过校准的信任似乎植根于系统的透明度及其对已知工作模式的遵循。因此，用户表现出更强的意愿，愿意随着时间的推移采用并维护HomeGenii（Q22, Q24: $p < .001$ ）。

控制与模糊性处理。HomeGenii的自然语言反馈循环被视为一种纠正规则（Q17: $p < .001$ ）的有效方式，同时保持了用户控制（Q18: $p < .01$ ）。其处理模糊请求（Q19, Q20: $p < .01$ ）的能力尤其受到重视，它可以从不明确的查询中提取相关示例来指导生成。

7 讨论

在本节中，我们将讨论 HomeGenii 设计的可持续性、可靠性和适用范围，这分别对应规则库的长期演进、其生成管道的鲁棒性，以及它在更广泛的自动化工具生态系统中的定位。

7.1 规则库的可扩展性与维护

规则库的可扩展性和长期维护是通过将抽象自动化逻辑与特定硬件实现解耦来维持的。扩展规则库可以通过一个引导过程实现，即用户验证过的高质量规则被泛化为模板并重新集成到知识库中。维护负担可以通过自动化的规则检查器进一步最小化，它确保新增或更新的条目在语法上保持正确并符合平台约束。为了适配新设备，系统仅需要更新家庭环境上下文，因为规则库使用的是参数化占位符而非固定的设备标识符。同样地，适应新的自动化平台或脚本语言是通过更新指令前导码中的目标模式来完成的，这使得相同的逻辑模式能够为不同的技术生态系统生成代码。

7.2 检索准确率及其对生成的影响

尽管我们的聚类后检索旨在寻找语义对齐的规则，但我们承认，偶尔检索到的规则可能在功能上接近但并非完全符合用户的意图。我们的实验分析表明，这种情况很少导致系统性的生成错误，原因有二。这很可能是因为同一簇内的示例在功能上相似，为大语言模型提供了足够的上下文信息，使其能够根据用户查询进行适当的泛化。为了进一步降低误导性检索的风险，系统可以轻松纳入多个Top-K检索到的规则作为参考，从而稀释任何单个不完美示例的影响。更重要的是，后续的规则检查器会根据平台模式验证生成的规则，并允许用户参与循环修正，为防范从检索阶段传播的错误提供了最终保障。

7.3 范围与边界

HomeGenii所解决的问题：HomeGenii有效地解决了将用户查询转化为增强型TAP规则的挑战，处理了控制逻辑和平台特定语法的复杂性，这些对于非专业用户来说是障碍。

正交问题：有两个正交问题未在HomeGenii中直接解决。首先，冲突检测与解决，即识别生成的或现有规则何时可能导致矛盾的设备状态或不安全交互，是文献中一个经过充分研究的验证问题 [4, 6, 54]。其次，高层目标消歧，即帮助用户阐明和细化模糊的自动化需求，属于需求获取的范畴 [45]。

集成路径：HomeGenii的架构为与这些正交组件集成提供了天然的切入点。用户参与循环验证步骤允许通过用户反馈来捕获和纠正冲突或误解。此外，该系统的输出可作为形式化验证工具的输入。因此，虽然HomeGenii专注于生成问题，但它与一个更广泛的、包含意图澄清和安全验证的自动化流程是兼容的。

8 结论

本文解决了智能家居中的一个关键可用性挑战：让不具备技术专长的用户无需编程技能即可创建增强型TAP规则。我们介绍了HomeGenii，这是首个为TAP规则生成设计的基于RAG的系统。通过检索专家整理的规则和环境上下文，HomeGenii能够输出高质量的增强型TAP规则，即使面对模糊的输入也能忠实地反映用户意图。在HomeGenii内部，我们提出了一种层次分类法来指导规则库构建，一种先聚类后搜索的检索机制来弥合语义鸿沟，以及一种压缩策略来减少令牌开销。评估结果表明，HomeGenii在生成增强型TAP规则方面达到了84%的显著准确率，相比没有使用RAG的系统提升了70%。作者已在<https://github.com/universeMorn/HomeGenii>公开提供了其代码和数据集。

致谢

本研究部分得到了国家自然科学基金项目（批准号：62472089）、江苏省网络与信息安全重点实验室项目（批准号：BM2003201）、教育部计算机与信息集成新技术重点实验室项目（批准号：93K-9）、新型软件技术与产业化协同创新中心以及东南大学大数据计算中心的支持。本文中的任何观点、发现、结论和建议均为作者个人观点，不一定代表资助机构的立场。

参考文献

- [1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altmenschmidt, Sam Altman, Shyamal Anadkat, 等人。2023. GPT-4技术报告。 *arXiv预印本 arXiv:2303.08774* (2023)。
- [2] Mohannad Alhanahnah, Clay Stevens, Bocheng Chen, Qiben Yan, 和 Hamid Bagheri。2022. IoTCom: 剖析物联网系统中的交互威胁。 *IEEE软件工程汇刊 (TSE)* 49, 4 (2022), 1523–1539。
- [3] Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George Bm Van Den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, 等人。2022. 通过从数万亿词元中检索来改进语言模型。载于 *国际机器学习会议 (ICML)*。PMLR, 2206–2240。
- [4] Z Berkay Celik, Patrick McDaniel, 和 Gang Tan。2018. Soteria: 自动化物联网安全与安保分析。载于 *2018 USENIX年度技术会议 (USENIX ATC)*。147–158。
- [5] Xiaohong Chen, Shi Chen, Zhi Jin, Han Bian, Zihan Chen, 和 Haotian Li。2025. 表达智能家居中的需求：终端用户最喜欢的方式是什么。 *ACM计算机-人机交互汇刊 (TOCHI)* 32, 2 (2025), 1–38。
- [6] Haotian Chi, Qiang Zeng, 和 Xiaojiang Du。2023. 在云平台多控制通道智能家居中检测和处物联网交互威胁。载于 *第32届USENIX安全研讨会论文集 (USENIX Security)*。1559–1576。
- [7] Meghan Clark, Prabal Dutta, 和 Mark W Newman。2016. 迈向智能家居的自然语言编程接口。载于 *2016年ACM普适与泛在计算国际联合会议：附加论文集*。49–52。
- [8] Leonardo De Moura 和 Nikolaj Bjørner。2008. Z3: 一个高效的SMT求解器。载于 *工具与算法构建与分析系统国际会议*。Springer, 337–340。
- [9] Jacob Devlin, Ming-Wei Chang, Kenton Lee, 和 Kristina Toutanova。2018. BERT: 用于语言理解的深度双向Transformer预训练。 *arXiv预印本 arXiv:1810.04805* (2018)。
- [10] Wenqi Fan, Yajuan Ding, Liangbo Ning, Shijie Wang, Hengyun Li, Dawei Yin, Tat-Seng Chua, 和 Qing Li。2024. 检索增强生成与大语言模型融合综述：迈向检索增强大语言模型。载于 *ACM SIGKDD知识发现与数据挖掘会议 (SIGKDD)*。6491–6501。

- [11] 张寅峰, 郭达雅, 唐都钰, 段楠, 冯晓成, 龚明, 寿林钧, 覃兵, 刘挺, 蒋大新, 等人。2020. CodeBERT: 一个用于编程和自然语言的预训练模型。arXiv预印本 arXiv:2002.08155 (2020)。[12] Thibault Févry, Livio Baldini Soares, Nicholas Fitzgerald, Eunsol Choi, 和 Tom Kwiatkowski。2020. 实体作为专家: 通过实体监督进行稀疏记忆访问。载于《2020年自然语言处理实证方法会议论文集 (EMNLP)》。4937–4951。[13] 高毅, 肖凯杰, 李福, 徐伟峰, 黄嘉明, 和董伟。2024. ChatIoT: 基于触发器-动作的物联网程序零代码生成。《ACM交互式、移动、可穿戴与普适技术汇刊 (IMWUT)》8, 3 (2024), 1–29。[14] DN Geary。1989. 混合模型: 推理及其在聚类中的应用。《皇家统计学会杂志A辑》152, 1 (1989), 126–127。[15] Edouard Grave, Armand Joulin, 和 Nicolas Usunier。2017. 使用连续缓存改进神经语言模型。载于《国际学习表征会议》。[16] Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, 和 Mingwei Chang。2020. 检索增强语言模型预训练。载于《国际机器学习会议 (ICML)》。PMLR, 3929–3938。[17] HomeAssistant。访问于 2025年10月。Home Assistant网站。[在线], <https://www.home-assistant.io>。[18] HomeAssistant。访问于 2025年10月。Python脚本集成。[在线], <https://www.home-assistant.io/integrations/python-script>。[19] IFTTT。访问于 2025年10月。使用筛选代码构建。[在线], <https://help.ifttt.com/hc/en-us/articles/360052451954-Building-with-filter-code>。[20] IFTTT。访问于 2025年10月。如何添加筛选代码。[在线], <https://help.ifttt.com/hc/en-us/articles/4406412223771-How-to-add-filter-code>。[21] IFTTT。访问于 2025年10月。IFTTT网站。[在线], <https://ifttt.com>。[22] Gautier Izacard, Patrick Lewis, Maria Lomeli, Lucas Hosseini, Fabio Petroni, Timo Schick, Jane Dwivedi-Yu, Armand Joulin, Sebastian Riedel, 和 Edouard Grave。2023. Atlas: 使用检索增强语言模型进行少样本学习。《机器学习研究杂志 (JMLR)》24, 251 (2023), 1–43。[23] Ankur Joshi, Saket Kale, Satish Chandel, 和 D Kumar Pal。2015. 李克特量表: 探索与解释。《英国应用科学与技术杂志》7, 4 (2015), 396。[24] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick SH Lewis, Ledell Wu, Sergey Edunov, 陈丹琦, 和叶文涛。2020. 用于开放域问答的密集段落检索。载于《2020年自然语言处理实证方法会议论文集 (EMNLP)》。6769–6781。[25] Urvashi Khandelwal, Omer Levy, Dan Jurafsky, Luke Zettlemoyer, 和 Mike Lewis。2020. 通过记忆实现泛化: 最近邻语言模型。载于《国际学习表征会议 (ICLR)》。[26] Evan King, 于浩翔, Sangsu Lee, 和 Christine Julien。2024. Sasha: 智能家居中使用大语言模型的创造性目标导向推理。《ACM交互式、移动、可穿戴与普适技术汇刊 (IMWUT)》8, 1 (2024), 1–38。[27] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, 叶文涛, Tim Rocktäschel, 等人。2020. 面向知识密集型自然语言处理任务的检索增强生成。《神经信息处理系统进展 (NeurIPS)》33 (2020), 9459–9474。[28] 李林蔚, 郭良辰, 何振英, 荆一楠, 和王希。2019. X-DMM: 快速且可扩展的基于模型的文本聚类。载于《AAAI人工智能会议论文集 (AAAI)》, 第33卷。4197–4204。[29] liftoff。[未注明日期]。Pyminifier工具。[在线], <https://github.com/liftoff/pyminifier>。访问于 2025年10月。[30] 刘立伟, 陈伟, 王涛, 王伟, 吴国权, 和魏峻。2023. 通过挖掘历史事件日志生成场景中心化TAP规则。载于《2023年IEEE国际Web服务会议 (ICWS)》。IEEE, 21–27。[31] 刘立伟, 王涛, 陈伟, 魏峻, 王伟, 和吴国权。2025. LEGO: 基于静态分析和大语言模型合成物联网设备组件。《ACM交互式、移动、可穿戴与普适技术汇刊 (IMWUT)》9, 2 (2025), 1–26。[32] 刘晓艺, 石映天, 余春, 高程, 杨天傲, 梁晨, 和史元春。2023. 理解智能家居自动化的原位编程。《ACM交互式、移动、可穿戴与普适技术汇刊 (IMWUT)》7, 2 (2023), 1–31。[33] Sunil Manandhar, Kevin Moran, Kaushal Kafle, 唐如昊, Denys Poshyvanyk, 和 Adwait Nadkarni。2020. 迈向智能家居的自然视角以进行实际安全与安保分析。载于《IEEE安全与隐私研讨会 (S&P)》。IEEE, 482–499。[34] 亚伯拉罕·哈罗德·马斯洛。1943. 人类动机理论。《心理学评论》50, 4 (1943), 370。[35] Andrea Mattioli 和 Fabio Paternò。2023. 一款用于在智能家居中创建、控制、推荐自动化的移动增强现实应用。《ACM人机交互汇刊 (PACMHCI)》7, MHCI (2023), 1–22。[36] Kamal Nigam, Andrew Kachites McCallum, Sebastian Thrun, 和 Tom Mitchell。2000. 使用期望最大化算法从标记和未标记文档中进行文本分类。《机器学习》39 (2000), 103–134。

- [37] Ori Ram, Yoav Levine, Itay Dalmedigos, Dor Muhlgay, Amnon Shashua, Kevin Leyton-Brown, and Yoav Shoham. 2023. 上下文检索增强语言模型. 计算语言学协会汇刊 (TACL) 11 (2023), 1316–1331. [38] Michaela R Reisinger, Sebastian Prost, Johann Schrammel, and Peter Fröhlich. 2023. 智能家居设计的用户需求: 维度与目标. 环境智能与以人为本计算杂志 14, 12 (2023), 15761–15780. [39] 任烁, 郭达雅, 卢帅, 周龙, 刘树杰, 唐都钰, Neel Sundaresan, 周明, Ambrosio Blanco, and 马帅. 2020. Codebleu: 一种代码合成的自动评估方法. arXiv preprint arXiv:2009.10297 (2020). [40] Stephen Robertson, Hugo Zaragoza, et al. 2009. 概率相关性框架: BM25及其超越. 信息检索基础与趋势 3, 4 (2009), 333–389. [41] 三星. 访问于 2025年10月. SmartThings. [在线], <https://www.samsung.com/us/smartthings>. [42] Emmanuel Senft, Michael Hagenow, Robert Radwin, Michael Zinn, Michael Gleicher, and Bilge Mutlu. 2021. 面向人机协作的情境化实时编程. In ACM用户界面软件与技术研讨会 (UIST). 613–625. [43] 徐旻俊. 2024. REPLUG: 检索增强黑盒语言模型. In 北美计算语言学协会会议 (NAACL). Association for Computational Linguistics. [44] 宋云鹏, 卞一恒, 王晓睿, and 蔡忠民. 2024. 从用户驱动事件学习生成自动化序列. ACM交互式、移动、可穿戴与普适技术汇刊 (IMWUT) 7, 4 (2024), 1–22. [45] Blase Ur, Elyse McManus, Melwyn Pak Yong Ho, and Michael L Littman. 2014. 智能家居中的实用触发-动作编程. In SIGCHI计算系统中的人为因素会议论文集 (CHI). 803–812. [46] 王悦, 王伟石, Shafiq Joty, and 许志钦. 2021. Codet5: 面向代码理解与生成的标识符感知统一预训练编码器-解码器模型. arXiv preprint arXiv:2109.00859 (2021). [47] 魏佳森, 王学志, Dale Schuurmans, Maarten Bosma, 夏飞, Ed Chi, Quoc V Le, 周丹尼, et al. 2022. 思维链提示激发大语言模型中的推理能力. 神经信息处理系统进展 (NeurIPS) 35 (2022), 24824–24837. [48] Frank Wilcoxon, SK Katti, Roberta A Wilcox, et al. 1963. Wilcoxon秩和检验与Wilcoxon符号秩检验的临界值与概率水平. Vol. 1. 美国氰胺公司 Pearl River, NY. [49] 吴宇怀, Markus N Rabe, DeLesley Hutchins, and Christian Szegedy. 2022. 记忆Transformer. arXiv preprint arXiv:2203.08913 (2022). [50] 殷建华 and 王建勇. 2014. 一种基于狄利克雷多项式混合模型的短文本聚类方法. In ACM SIGKDD知识发现与数据挖掘会议 (SIGKDD). 233–242. [51] 于浩翔, 华杰, and Christine Julien. 2021. 分析IFTTT配方以研究人类如何使用物联网设备. In 第19届ACM嵌入式网络传感器系统会议论文集 (SenSys). 537–541. [52] Imam Nur Bani Yusuf, 蒋凌霄, and 罗大卫. 2022. 使用领域自适应序列到序列学习准确生成触发-动作程序. In 第30届IEEE/ACM国际程序理解会议论文集 (ICPC). 99–110. [53] Zapier. 访问于 2025年10月. Zapier 代码集成. [在线], <https://zapier.com/apps/code/integrations>. [54] 张乐凡, 何伟佳, Jesse Martinez, Noah Brackenburg, 卢山, and Blase Ur. 2019. AutoTap: 使用LTL属性合成与修复触发-动作程序. In 第41届IEEE/ACM国际软件工程会议论文集 (ICSE). 281–291. [55] 张乐凡, 何伟佳, Olivia Morkved, Valerie Zhao, Michael L Littman, 卢山, and Blase Ur. 2020. Trace2tap: 从行为轨迹合成触发-动作程序. ACM交互式、移动、可穿戴与普适技术汇刊 (IMWUT) 4, 3 (2020), 1–26. [56] 张乐凡, 周赛勒斯, Michael L Littman, Blase Ur, and 卢山. 2023. 帮助用户调试触发-动作程序. ACM交互式、移动、可穿戴与普适技术汇刊 (IMWUT) 6, 4 (2023), 1–32. [57] 钟泽轩, 雷涛, and 陈丹琦. 2022. 使用记忆增强训练语言模型. In 2022年自然语言处理经验方法会议论文集 (EMNLP).

A 用户研究的补充数据

A.1 HomeGenii 快速入门

HomeGenii - 快速开始

目的：本教程帮助您理解如何使用HomeGenii

主界面区域：

HomeGenii

Choose LUT

QFT 5

Target thermostat

Target location

Choose location

From page 30

Visual Plan

Left

居中

底部

- 左侧：平面图 + 设备。 - 中间：生成的自动化（高 / 中 / 低）。 - 底部：输入框 + 操作按钮（看起来不错 / 做出更改 / 取消）。

HomeGenii 在三个抽象级别生成的规则：

- 高层意图：规则的自然语言目的 - 中层计划：以结构化、人类可读的格式呈现的技术逻辑 - 底层实现：实际可执行的规则

操作说明

1. 在输入框中键入请求。 2. 等待系统生成三级输出。 3. 阅读高层意图和中层计划（这些解释了将要发生的事情）。 4. 决定：

看起来不错做出更改

→ 丢弃

快速检查

1. 根据教程，哪个概念代表“用户目标”？

☐ 高层意图 ☐ 中层计划 ☐ 底层实现

2. 以下哪项是“中层计划”的示例？

☐ 让客厅变得舒适。 ☐ 调暗灯具，设置恒温器，启动香薰机。

☐ 可执行的TAP代码。

3. 如果生成的输出看起来正确，您应该采取什么动作？

☐ 看起来不错 ☐ 做出更改 ☐ 取消

A.2 参与者人口统计信息

表 8. 参与者人口统计信息。

PID	Age	性别	职业/行业	最高学历	智能家居使用经验	使用频率
P1	25-34岁	女性	软件工程师	硕士	2+ 年	每日
P2	18-24岁	Male	学生	学士学位	从未使用	从不
P3	35-44岁	Male	产品经理	硕士	1年	每周
P4	25-34岁	女性	用户体验设计师	硕士	2+ 年	每日
P5	45-54岁	Male	教师	学士学位	2+ 年	偶尔
P6	18-24岁	女性	学生	高中或以下	从未使用	从不
P7	25-34岁	Male	数据分析师	学士学位	1年	每周
P8	35-44岁	女性	市场营销经理	学士学位	2+ 年	每周
P9	25-34岁	Male	自由职业者	硕士	6个月	偶尔
P10	55-64岁	Male	退休人员	高中或以下	6个月	偶尔
P11	18-24岁	女性	学生	学士学位	1年	每周
P12	35-44岁	Male	工程师	博士学位	2+ 年	每日
P13	25-34岁	女性	研究员	博士学位	1年	每日
P14	45-54岁	女性	护士	学士学位	6个月	每周
P15	18-24岁	Male	学生	学士学位	从未使用	偶尔